

Team Note of 찬우없는방

조찬우, 전역, 축하합니다

Compiled on August 31, 2026

Contents

1 Basic Implementation	1
1.1 Main Template	1
2 Math	1
2.1 Basic Arithmetic	1
2.2 Binomial Coefficient	2
2.3 Chinese Remainder Theorem	2
2.4 FFT & NTT	2
2.5 Bostan-Mori	3
2.6 Linear Sieve	3
2.7 Miller-Rabin & Pollard-Rho	3
2.8 XOR Basis	3
2.9 Floor Sum	4
2.10 Xuduh Sieve	4
2.11 Discrete Log	4
2.12 Multiplicative Order	4
2.13 Power Tower	5
2.14 Gaussian Elimination	5
3 Data Structure	5
3.1 Erasable Priority Queue	5
3.2 Non-Recursive Segment Tree	5
3.3 Inversion counting	6
3.4 1D & 2D Fenwick Tree	6
3.5 Merge Sort Tree	6
3.6 Persistent Segment Tree	6
3.7 Link-Cut Tree	6
3.8 Li Chao Tree	8
3.9 DSU (Weighted)	8
3.10 Trie	8
3.11 Sparse Table	9
3.12 Mo's Algorithm	10
4 Graph	10
4.1 Bellman Ford	10
4.2 LCA	10
4.3 HLD	10
4.4 Centroid Decomposition	11
4.5 Bipartite Matching	11
4.6 Hungarian	12
4.7 Dinic	12
4.8 MCMF	13
4.9 Circulation	13
4.10 SCC	13
4.11 2-sat	13

4.12 BCC	14
4.13 Find 3 or 4 cycle	14
4.14 Push Relabel	15
4.15 Gomory Hu Tree	15
5 DP Optimization	15
5.1 Convex Hull Trick	15
5.2 Linear CHT	16
5.3 D&C optimization	16
5.4 Monotone Queue optimization	16
5.5 Aliens Trick	16
5.6 Sum Over Subsets	16
5.7 Berlekamp Massey	17
6 Geometry	17
6.1 Geometry Template	17
6.2 Closest Pair	17
6.3 Convex Hull	18
6.4 Convex Hull 3d	18
6.5 Rotating Calipers	18
6.6 Point in Convex Polygon	18
6.7 Point in Polygon	18
6.8 Sort Points	18
6.9 Linear Minkowski Sum	18
6.10 Smallest Enclosing Circle	19
6.11 Geometric Intersections	19
6.12 Half Plane Intersection	19
6.13 Manhattan MST	20
7 String	20
7.1 Aho-Corasick	20
7.2 Hashing	20
7.3 KMP	21
7.4 Manacher	21
7.5 Suffix Array	21
7.6 Z-algorithm	21
7.7 Eertree	21
7.8 Suffix Automaton	21
8 STL & pbds	22
8.1 Hash map (pb_ds)	22
8.2 Ordered Set (pb_ds)	22
8.3 Permutation & Combination	22
8.4 Priority Queue (pb_ds)	22
8.5 Rope	22
9 Misc	22
9.1 Custom Hash	22
9.2 Fast I/O	23
9.3 Random	23
9.4 Ternary Search	23
9.5 Some tricks	23

10 Checklist + Useful Info	24
10.1 Highly Composite Numbers, Large Prime	24
10.2 Useful Stuff	24
10.3 자주 쓰이는 문제 접근법	25
10.4 DP 최적화 접근	25
10.5 Graph Matching(Graph with $ V \leq 500$)	25
10.6 Mincut 모델링	25
10.7 LP & Flow Duality	25

1 Basic Implementation

1.1 Main Template

```
// alias gpp='g++ -std=c++20 -Wall -Wshadow -O2 -DLOCAL
-Wno-unused-variable -fsanitize=address,undefined'
#include <bits/stdc++.h> // #include <bits/extc++.h> for pbds
#pragma GCC optimize ("O3,unroll-loops")
#pragma GCC target ("avx,avx2,fma")
#define fastio cin.tie(0)->sync_with_stdio(0)
#define all(x) (x).begin(),(x).end()
#define rall(x) (x).rbegin(),(x).rend()
#define compress(v) sort(all(v)), v.erase(unique(all(v)),
v.end())
#define sz(x) (int)(x).size()
using namespace std;
using ll = long long;
```

2 Math

2.1 Basic Arithmetic

```
ll modmul(ll a, ll b, ll m) { return (ll)((__int128)a*b %
m); }
ll modpow(ll b, ll e, ll m) {
    ll ans = 1;
    for (; e; b = modmul(b, b, m), e /= 2)
        if (e & 1) ans = modmul(ans, b, m);
    return ans;
}
ll xgcd(ll a, ll b, ll &x, ll &y) {
    if (!b) return x = 1, y = 0, a;
    ll x1, y1, g = xgcd(b, a % b, x1, y1);
    return x = y1, y = x1 - a / b * y1, g;
}
ll modinv(ll a, ll m) {
    ll x, y;
    ll g = xgcd(a, m, x, y);
    if (g != 1) return -1;
    return (x%m + m) % m;
}
ll phi(ll n) {
    ll res = n;
    for (ll i = 2; i*i <= n; i++) if (n%i == 0) {
        res -= res / i;
        while (n%i == 0) n /= i;
    } if (n > 1) res -= res / n;
    return res;
}
```

2.2 Binomial Coefficient

Time Complexity: Init: $O(MAXN)$, Query: $O(1)$

```
// when M is big prime; Init: O(MAXN), Query: O(1)
// modmul, modpow
const int M = 1e9+7, MAXN = 4000000;
ll fac[MAXN+5], finv[MAXN+5];
void init() {
    fac[0] = 1;
    for (int i = 1; i <= MAXN; i++)
        fac[i] = modmul(fac[i-1], i, M);
    finv[MAXN] = modpow(fac[MAXN], M-2, M);
    for (int i = MAXN-1; i >= 0; i--)
        finv[i] = modmul(finv[i+1], i+1, M);
}
ll nCk(int n, int k) {
    if (n < k || k < 0) return 0;
    ll r = modmul(fac[n], finv[n-k], M);
    return modmul(r, finv[k], M);
}
```

2.3 Chinese Remainder Theorem

Usage: Solve system of linear congruences.

Time Complexity: $O(\log N)$

```
// xgcd
pair<ll,ll> CRT(ll a1, ll m1, ll a2, ll m2) {
    ll x, y, g = xgcd(m1, m2, x, y);
    if ((a2 - a1) % g) return { -1, -1 };
    ll md = m2 / g, k = (a2 - a1) / g * md * (x % md) % md;
    return { a1 + (k < 0 ? k + md : k) * m1, m1 / g * m2 };
}
pair<ll,ll> CRT(const vector<ll>& a, const vector<ll>& m) {
    ll ra = a[0], rm = m[0];
    for (int i = 1; i < (int)m.size(); i++) {
        auto [aa, mm] = CRT(ra, rm, a[i], m[i]);
        if (mm == -1) return { -1, -1 };
        ra = aa; rm = mm;
    }
    return { ra, rm };
}
```

2.4 FFT & NTT

Usage: Fast Fourier/Number Theoretic Transform for convolutions.

Time Complexity: $O(N \log N)$

```
template <ll M = 998244353>
struct Modint {
    using V = long long; V val;
    Modint() : val(0) {} Modint(auto y) : val(y % M) {}
    operator V() const { return val; }
    Modint operator-() const { return Modint() -= *this; }
    Modint operator+(auto rhs) const { return Modint(*this) += rhs; }
    Modint operator-(auto rhs) const { return Modint(*this) -= rhs; }
    Modint operator*(auto rhs) const { return Modint(*this) *= rhs; }
    Modint operator/(auto rhs) const { return Modint(*this) /= rhs; }
}
```

```
Modint &operator+=(Modint rhs) { val += rhs.val; if (val >= M) val -= M; return *this; }
Modint &operator-=(Modint rhs) { val -= rhs.val; if (val < 0) val += M; return *this; }
Modint &operator*=(Modint rhs) { val = val * rhs.val % M; return *this; }
Modint &operator/=(Modint rhs) { val = val * rhs.inv() % M; return *this; }
Modint inv() { return inv(val, M); }
V inv(ll x, ll m) { return x > 1 ? m - inv(m % x, x) * m / x : 1; }
Modint pow(auto y) {
    if (y == 0) return Modint(1);
    if (y < 0) return Modint(val).inv().pow(-y);
    Modint ans(1), x(val);
    while (y) { if (y % 2) ans *= x; x *= x; y /= 2; }
    return ans;
}
friend std::ostream &operator<<(std::ostream &os, const Modint<M> &x) { return os << x.val; }
friend std::istream &operator>>(std::istream &is, Modint<M> &x) { ll v; is >> v; x = v; return is; }
};
using Mint = Modint<998244353>;
using Poly = vector<Mint>;
```

```
struct FPS {
    Poly coef;
    FPS(Poly a) : coef(a) {}
    FPS(int n = 0) { coef.resize(1); coef[0] = Mint(n); }
    int size() { return coef.size(); }
    int deg() { return size() - 1; }
    Mint& operator[](int index) { assert(index < coef.size()); return coef[index]; }
    FPS operator+(auto f) {
        FPS res; res.resize(max(size(), f.size()));
        for (int i = 0; i < size(); i++) res.coef[i] += coef[i];
        for (int i = 0; i < f.size(); i++) res.coef[i] += f.coef[i];
        return res;
    }
    FPS operator-(auto f) {
        FPS res; res.resize(max(size(), f.size()));
        for (int i = 0; i < size(); i++) res.coef[i] += coef[i];
        for (int i = 0; i < f.size(); i++) res.coef[i] -= f.coef[i];
        return res;
    }
    FPS operator*(auto f) {
        Poly nc = polymul(coef, f.coef);
        nc.resize(size() + f.size() - 1); return FPS(nc);
    }
    FPS &operator+=(auto f) { return *this = *this + f; }
    FPS &operator-=(auto f) { return *this = *this - f; }
    FPS &operator*=(auto f) { return *this = *this * f; }
    void resize(int sz) { while (size() < sz) coef.push_back(Mint(0)); while (size() > sz) coef.pop_back(); }
}
```

```
void shrink() { while (size() > 1 && coef.back() == 0) coef.pop_back(); }
FPS power(ll k) {
    FPS res(1), f = *this; res[0] = 1;
    while (k) { if (k % 2) res = res * f; f = f * f; k /= 2; }
    return res;
}
FPS inv() {
    assert(coef[0] != 0); FPS g(Poly{Mint(coef[0]).inv()});
    two(2);
    for (int sz = 1; sz < size(); sz *= 2) {
        FPS f(*this); f.resize(sz * 2);
        FPS tmp = g * f; tmp.resize(sz * 2);
        g = g * (two - tmp); g.resize(sz * 2);
    }
    g.resize(size()); return g;
}
FPS log() {
    assert(coef[0] != 0); FPS g = differentiate() * inv();
    g.resize(size()); g = g.integrate(); g.resize(size());
    return g;
}
FPS differentiate() {
    FPS res; res.resize(size() - 1);
    for (int i = 1; i < size(); i++) res.coef[i - 1] = coef[i] * i;
    return res;
}
FPS integrate() {
    FPS res; res.resize(size() + 1);
    for (int i = 1; i <= size(); i++) res.coef[i] = coef[i - 1] * Mint(i).inv();
    return res;
}
FPS exp() {
    assert(coef[0] == 0); FPS g(1), one(1);
    for (int sz = 1; sz < 2 * size(); sz *= 2) {
        FPS f(*this); f.resize(sz * 2);
        g = g * (f + one - g.log()); g.resize(sz * 2);
    }
    g.resize(size()); return g;
}
void print() { for (int i = 0; i < size(); i++) cout << coef[i] << ' '; cout << endl; }
Mint evaluate(Mint x) {
    Mint res(0);
    for (int i = size() - 1; i >= 0; i--) { res *= x; res += coef[i]; }
    return res;
}
private:
void ntt(Poly &P, bool inv, Mint g) {
    int n = P.size(); if (n == 1) return;
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1; for (; j & bit; bit >= 1) j ^= bit;
        j ^= bit; if (i < j) swap(P[i], P[j]);
    }
    vector<Mint> w(n / 2); w[0] = 1;
    for (int i = 1; i < n / 2; i++) w[i] = w[i - 1] * g;
```

```

for (int i = 1; i < n; i <= 1) {
    int nd = n / (2 * i);
    for (int j = 0; j < n; j += i < 1) {
        for (int k = 0; k < i; k++) {
            Mint tmp = P[i + j + k] * w[nd * k];
            P[i + j + k] = P[j + k] - tmp; P[j + k] += tmp;
        }
    }
}
if (inv) {
    Mint invn = Mint(n).inv();
    for (int i = 0; i < n; i++) P[i] *= invn;
}
}
Poly polymul(Poly v1, Poly v2) {
    ll n1 = v1.size(), n2 = v2.size(), N = 1;
    while (N <= n1 + n2 - 1) N *= 2;
    v1.resize(N); v2.resize(N);
    Mint g = Mint(3).pow(998244353 / N);
    ntt(v1, false, g); ntt(v2, false, g);
    Poly res; res.resize(N);
    for (int i = 0; i < N; i++) res[i] = v1[i] * v2[i];
    ntt(res, true, g.inv()); return res;
}
};

```

2.5 Bostan-Mori

Time Complexity: $O(d \log d \log k)$

```

Mint bostan_mori(FPS P, FPS Q, ll k) {
    // return [x^k] P/Q
    while (k) {
        FPS Q_(Q.coef);
        for (int i = 1; i <= Q_.deg(); i+=2) {
            Q_.coef[i] *= Mint(-1);
        }
        P *= Q_; Q *= Q_;
        Poly tmp;
        for (int i = k%2; i <= P.deg(); i+=2) {
            tmp.push_back(P.coef[i]);
        }
        P = FPS(tmp);
        tmp.clear();
        for (int i = 0; i <= Q_.deg(); i+=2) {
            tmp.push_back(Q_.coef[i]);
        }
        Q = FPS(tmp);
        k /= 2;
    }
    return P.evaluate(0) / Q.evaluate(0);
}

```

2.6 Linear Sieve

Usage: Find primes and multiplicative functions in linear time.

Time Complexity: $O(N)$

```

struct Sieve {
    // sp: 최소 소인수, e: i의 최소 소인수 지수, phi: 오일러 피
    함수(1~i 중 i와 서로소인 개수), mu: 뫼비우스 함수, tau: 약수
    개수, sigma: 약수의 합

```

```

vector<int> sp, e, phi, mu, tau, sigma, tmp, primes;
Sieve(int n) : sp(n+1), e(n+1), phi(n+1), mu(n+1),
    tau(n+1), sigma(n+1), tmp(n+1) {
    phi[1] = mu[1] = tau[1] = sigma[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!sp[i]) {
            sp[i]=i; primes.push_back(i);
            e[i]=1; phi[i]=i-1; mu[i]=-1; tau[i]=2;
            sigma[i]=tmp[i]=i+1;
        }
        for (int p : primes) {
            if (i*p > n || p > sp[i]) break;
            int m = i*p; sp[m] = p;
            if (i%p == 0) {
                e[m] = e[i]+1; phi[m] = phi[i]*p; mu[m] = 0;
                tau[m] = tau[i]/(e[i]+1)*(e[m]+1);
                tmp[m] = tmp[i]*p+1; sigma[m] =
                    sigma[i]/tmp[i]*tmp[m];
                break;
            }
            e[m] = 1; phi[m] = phi[i]*(p-1); mu[m] = -mu[i];
            tau[m] = tau[i]*2; tmp[m] = p+1; sigma[m] =
                sigma[i]*(p+1);
        }
    }
}
};

```

2.7 Miller-Rabin & Pollard-Rho

Usage: Primality test and integer factorization.

Time Complexity: $O(\log^3 N)/O(N^{1/4})$

```

// modmul, modpow
bool isPrime(ll n) {
    if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
    ll A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
        s = __builtin_ctzll(n-1), d = n >> s;
    for (ll a : A) { // ^ count trailing zeroes
        ll p = modpow(a%n, d, n), i = s;
        while (p != 1 && p != n - 1 && a % n && i--)
            p = modmul(p, p, n);
        if (p != n-1 && i != s) return 0;
    }
    return 1;
}
ll pollard(ll n) {
    auto f = [n](ll x) { return modmul(x, x, n) + 3; };
    ll x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    while (t++ % 40 || __gcd(prd, n) == 1) {
        if (x == y) x = ++i, y = f(x);
        if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q;
        x = f(x), y = f(f(y));
    }
    return __gcd(prd, n);
}
vector<ll> factor(ll n) {
    if (n == 1) return {};
    if (isPrime(n)) return {n};
    ll x = pollard(n);

```

```

    auto l = factor(x), r = factor(n / x);
    l.insert(l.end(), r.begin(), r.end());
    return l;
}
vector<ll> res = factor(N); // factor of N

```

2.8 XOR Basis

```

/* using Mask = bitset<MAX_N>; */
template <typename T = ll, int B = 62> struct Basis {
    T b[B]{}; int sz = 0;
    /* Mask mask[B]{}; */
    // 기저 원소 삽입, 0(B)
    bool insert(T x /*, int id*/) {
        /* Mask cur; cur.set(id); */
        for (int i = B-1; i >= 0; i--) {
            if ((x >> i & 1)) continue;
            if (!b[i]) {
                b[i] = x; sz++;
                /* mask[i] = cur; */
                return true;
            }
            x ^= b[i];
            /* cur ^= mask[i]; */
        }
        return false;
    }
    // 생성 가능 여부, 0(B)
    bool can(T x /*, Mask &res*/) const {
        /* res.reset(); */
        for (int i = B-1; i >= 0; i--) {
            if (x >> i & 1) {
                if (!b[i]) return false;
                x ^= b[i];
                /* res ^= mask[i]; */
            }
        }
        return true;
    }
    // 만들 수 있는 최댓값, 0(B)
    T max_xor(T init = 0 /*, Mask &res*/) const {
        /* res.reset(); */
        for (int i = B-1; i >= 0; i--) {
            if ((init ^ b[i]) > init) {
                init ^= b[i];
                /* res ^= mask[i]; */
            }
        }
        return init;
    }
    // 0 제외 최솟값, 0(B)
    T min_xor() const {
        for (int i = 0; i < B; i++) if (b[i]) return b[i];
        return 0;
    }
    // x XOR 최솟값 (생성 가능 시 0), 0(B)
    T reduce(T x) const {
        for (int i = B - 1; i >= 0; i--) {
            if (x >> i & 1) x ^= b[i];
        }
    }
}

```

```

    }
    return x;
}
// 기약 사다리꼴(RREF) 기저 목록, 0(B^2)
vector<T> rref() const {
    T tmp[B]; copy(begin(b), end(b), tmp);
    for (int i = 0; i < B; i++) {
        if (!tmp[i]) continue;
        for (int j = i + 1; j < B; j++) {
            if (tmp[j] >> i & 1) tmp[j] ^= tmp[i];
        }
    }
    vector<T> rb;
    for (int i = 0; i < B; i++) if (tmp[i])
        rb.push_back(tmp[i]);
    return rb;
}
// k번째 최소 XOR 합 (1-idx, 0 포함), 0(B^2)
T kth(ll k) const {
    if (k > (1ULL << sz)) return -1;
    auto rb = rref();
    T res = 0;
    for (int i = 0; i < sz; i++) {
        if ((k-1) >> i & 1) res ^= rb[i];
    }
    return res;
}
// x의 순위 (1-idx, 0 포함, 불가능 시 -1), 0(B^2)
ll rank(T x) const {
    if (!can(x)) return -1;
    auto rb = rref();
    ll res = 0;
    for (int i = 0; i < sz; i++) {
        if (x >= (x ^ rb[i])) {
            x ^= rb[i];
            res |= (1LL << i);
        }
    }
    return res + 1;
}
// 기저 병합, 0(B^2)
void merge(const Basis& o) {
    for (int i = 0; i < B; i++) if (o.b[i]) insert(o.b[i]);
}
};

```

2.9 Floor Sum

Usage: $\sum_{k=1}^n \lfloor n/k \rfloor$

```

// count of numbers such that n/i = k -> n/k - n/(k+1)
ll floor_sum(ll n) {
    ll sum = 0;
    for (ll i = 1, last; i <= n; i = last + 1) {
        last = n / (n / i);
        sum += (n / i) * (last - i + 1);
        // n / x yields the same value for i <= x <= last.
    }
    return sum;
}

```

2.10 Xuduh Sieve

Usage: Calculate prefix sum of multiplicative function

Time Complexity: $O(N^{3/4})$

```

// Xuduh Sieve: multiplicative function f의 prefix sum을
// 0(N^{2/3})에.
// pf(n) : f의 prefix sum (n이 작을 때, 선형체로 미리 계산)
// pg(n) : g의 prefix sum (g는 f*g가 쉬운 함수. 보통 1(x)=1
// -> n)
// pfg(n) : (f*g)의 prefix sum
// xudyh_sieve sieve(th, pf, pg, pfg); -> sieve.query(N)
/* e(x) = [x==1], 1(x) = 1, id_k(x) = x^k
mu: mobius function, id(x) = x
phi: euler totient function
sigma_k: sum of k-th power of divisors
sigma = sigma_1, d = tau = sigma_0
sigma_k = id_k * 1 | sigma = id * 1
id_k = sigma_k * mu | id = sigma * mu
e = 1 * mu | d = 1 * 1 | 1 = d * mu
phi * 1 = id | phi = id * mu | sigma = phi * d
g = f * 1 iff f = g * mu */
template<class T, class F1, class F2, class F3>
struct xudyh_sieve{
    T th; // threshold, 2(single query) ~ 5 * MAXN^2/3
    F1 pf; F2 pg; F3 pfg;
    // prefix sum of f(up to th), g(easy to calc), f*g(easy to
    // calc)
    unordered_map<T, T> mp; // f * g means dirichlet conv.
    xudyh_sieve(T th, F1 pf, F2 pg, F3
    pfg):th(th),pf(pf),pg(pg),pfg(pfg){}
    // Calculate the preix sum of a multiplicative f up to n
    T query(T n){ // 0(n^2/3)
        if(n <= th) return pf(n); if(mp.count(n)) return mp[n];
        T res = pfg(n);
        for(T low = 2, high = 2; low <= n; low = high + 1){
            high = n / (n / low);
            res -= (pg(high) - pg(low - 1)) * query(n / low); // MOD
        }
        return mp[n] = res / pg(1); //Pow(pg(1),MOD-2)?
    }
};

```

2.11 Discrete Log

Time Complexity: $O(\sqrt{P} \log P)$

```

int primitive_root(int p) { // p is prime
    vector<int> fact;
    int phi = p-1, n = phi;
    for (int i = 2; i*i <= n; i++) {
        if (n%i == 0) {
            fact.push_back(i);
            while (n%i == 0) n /= i;
        }
    }
    if (n > 1) fact.push_back(n);
    for (int res = 2; res <= p; res++) { // this loop will run
        at most (logp ^ 6) times i.e. until a root is found
        bool ok = true;
        // check if this is a primitive root modulo p
    }
}

```

```

    for (int i = 0; i < sz(fact) && ok; i++) ok &=
        modpow(res, phi / fact[i], p) != 1;
    if (ok) return res;
}
return -1;
}
// returns any or all numbers x such that x ^ k = a (mod m)
// existence: a = 0 is trivial, and if a > 0: a ^ (phi(m) /
// gcd(k, phi(m))) == 1 mod m
// if solution exists, then number of solutions = gcd(k,
// phi(m)).
// here m is prime, but it will work for any integer which
// has a primitive root
int discrete_root(int k, int a, int m) {
    if (a == 0) return 1;
    int g = primitive_root(m), phi = m-1; // m is prime
    // run baby step-giant step
    int sq = (int)sqrt(m+.0) + 1;
    vector<pair<int,int>> dec(sq);
    for (int i = 1; i <= sq; i++) dec[i-1] = { modpow(g, 1LL *
    i * sq % phi * k % phi, m), i };
    sort(all(dec)); int ans = -1;
    for (int i = 0; i < sq; i++) {
        int my = modpow(g, 1LL * i * k % phi, m) * 1LL * a % m;
        auto it = lower_bound(all(dec), make_pair(my, 0));
        if (it != dec.end() && it->first == my) {
            ans = it->second * sq - i; break;
        }
    }
    if (ans == -1) return -1; // no solution
    int d = (m-1) / __gcd(k, m-1);
    return modpow(g, ans%d, m);
    /* // for all possible answers
    int d = (m-1) / __gcd(k, m-1);
    vector<int> va;
    for (int i = ans%d; i < m-1; i += d) va.push_back(modpow(g,
    cur, m));
    sort(all(va));
    // assert(ans.size() == __gcd(k, m - 1))
    return va; */
}

```

2.12 Multiplicative Order

Usage: Find the minimum positive k s.t. $a^k \equiv 1 \pmod{m}$

```

ll phi(ll n);
ll get_order(ll a, ll mod) {
    if (__gcd(a, mod) != 1) return -1;
    ll m = phi(mod), p = m, ans = 2e18;
    if (power(a, p, mod) == 1) ans = p;
    vector<ll> fac;
    for (ll i = 2; i*i <= m; i++) {
        while(m%i == 0) m /= i, fac.push_back(i);
    }
    if (m > 1) fac.push_back(m);
    for (auto x : fac) {
        if (power(a, p/x, mod) == 1) p /= x, ans = p;
    }
    assert(ans != 2e18); return ans;
}

```

2.13 Power Tower

```
ll phi(ll n);
ll exmul(ll a, ll b, ll m) {
    if (a == 0 || b == 0) return 0;
    if (a*b >= m) return (a*b)%m + m;
    return a*b;
}
ll expow(ll b, ll e, ll m) {
    if (m == 1) return 1;
    ll ans = 1;
    for (; e; b = exmul(b, b, m), e /= 2)
        if (e & 1) ans = exmul(ans, b, m);
    return ans;
}
ll power_tower(const vector<ll>& a, int idx, ll m) {
    if (idx == sz(a) || m == 1) return 1;
    ll p = phi(m), exp = power_tower(a, idx+1, p);
    return expow(a[idx], exp, m);
}
// cout << power_tower(a, 0, mod) % mod;
```

2.14 Gaussian Elimination

```
// modpow, modinv
struct Gauss {
    const ll M = 1000000007;
    // return: 소거 성공 여부(false면 역행렬/해 없음)
    bool rref(vector<vector<ll>> &a, vector<vector<ll>> &b,
    int n) {
        for (int c = 0; c < n; c++) {
            int piv = c;
            while (piv < n && a[piv][c] == 0) piv++;
            if (piv == n) return false;
            swap(a[c], a[piv]); swap(b[c], b[piv]);
            ll d = modinv(a[c][c]);
            for (int j = c; j < n; j++) a[c][j] = a[c][j] * d % M;
            for (int j = 0; j < sz(b[c]); j++) b[c][j] = b[c][j] * d % M;
            for (int i = 0; i < n; i++) if (i != c) {
                ll f = a[i][c];
                if (f == 0) continue;
                for (int j = c; j < n; j++)
                    a[i][j] = (a[i][j] - f * a[c][j] % M + M) % M;
                for (int j = 0; j < sz(b[i]); j++)
                    b[i][j] = (b[i][j] - f * b[c][j] % M + M) % M;
            }
        }
        return true;
    }
    // Ax=b 해 (없으면 {})
    vector<ll> solve(vector<vector<ll>> a) {
        int n = sz(a);
        vector<vector<ll>> A(n, vector<ll>(n)), B(n,
        vector<ll>(1));
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) A[i][j] = a[i][j];
            B[i][0] = a[i][n];
        }
        if (!rref(A, B, n)) return {};
        vector<ll> x(n);
```

```
    for (int i = 0; i < n; i++) x[i] = B[i][0];
    return x;
}
// 역행렬 (없으면 {})
vector<vector<ll>> inverse(vector<vector<ll>> a) {
    int n = sz(a);
    vector<vector<ll>> b(n, vector<ll>(n));
    for (int i = 0; i < n; i++) b[i][i] = 1;
    if (!rref(a, b, n)) return {};
    return b;
}
// 행렬식
ll det(vector<vector<ll>> a) {
    int n = sz(a); ll res = 1;
    for (int c = 0; c < n; c++) {
        int piv = c;
        while (piv < n && a[piv][c] == 0) piv++;
        if (piv == n) return 0;
        if (piv != c) { swap(a[c], a[piv]); res = (M - res) % M; }
        res = res * a[c][c] % M;
        ll d = modinv(a[c][c]);
        for (int j = c; j < n; j++) a[c][j] = a[c][j] * d % M;
        for (int i = c + 1; i < n; i++) {
            ll f = a[i][c];
            if (f == 0) continue;
            for (int j = c; j < n; j++)
                a[i][j] = (a[i][j] - f * a[c][j] % M + M) % M;
        }
    }
    return res;
}
// rank
int rank(vector<vector<ll>> a) {
    int n = sz(a), m = sz(a[0]), r = 0;
    for (int c = 0; c < m; c++) {
        int piv = r;
        while (piv < n && a[piv][c] == 0) piv++;
        if (piv == n) continue;
        swap(a[r], a[piv]);
        ll d = modinv(a[r][c]);
        for (int j = c; j < m; j++) a[r][j] = a[r][j] * d % M;
        for (int i = r + 1; i < n; i++) {
            ll f = a[i][c];
            if (f == 0) continue;
            for (int j = c; j < m; j++)
                a[i][j] = (a[i][j] - f * a[r][c] % M + M) % M;
        }
        r++;
    }
    return r;
}
};
```

3 Data Structure

3.1 Erasable Priority Queue

```
template <typename T = int, typename Compare = less<T>>
struct ErasablePQ {
```

```
    priority_queue<T, vector<T>, Compare> q, del;
    void flush() {
        while (!del.empty() && !q.empty() && q.top() ==
        del.top()) {
            q.pop(); del.pop(); }
    }
    void push(const T& x) { q.push(x); flush(); }
    void erase(const T& x) { del.push(x); flush(); }
    void pop() { flush(); if (!q.empty()) q.pop(); flush(); }
    const T& top() { flush(); return q.top(); }
    int size() const { return int(q.size() - del.size()); }
    bool empty() { return q.size() == del.size(); }
};
```

3.2 Non-Recursive Segment Tree

```
template<typename Node> struct SegTree {
    int n, siz; Node e; // 항등원
    vector<Node> tr;
    function<Node(Node, Node)> fn;
    SegTree(int n, const Node& e, auto fn) : n(n),
    siz(1<<(__lg(n)+1)), e(e), tr(siz<<1, e), fn(fn) {}
    SegTree(const vector<Node>& v, const Node& e, auto fn) :
    n(sz(v)), siz(1<<(__lg(n)+1)), e(e), tr(siz<<1, e), fn(fn)
    {
        for (int i = 0; i < n; i++) tr[i+siz] = v[i];
        for (int i = siz-1; i > 0; i--) tr[i] = fn(tr[i<<1],
        tr[i<<1 | 1]);
    }
    void add(int i, const Node& val) {
        tr[i += siz] += val;
        while (i >= 1) tr[i] = fn(tr[i<<1], tr[i<<1 | 1]);
    }
    void update(int i, const Node& val) {
        tr[i += siz] = val;
        while (i >= 1) tr[i] = fn(tr[i<<1], tr[i<<1 | 1]);
    }
    Node query(int i) { return tr[i + siz]; }
    Node query(int l, int r) {
        Node L = e, R = e;
        for (l += siz, r += siz; l <= r; l >= 1, r >= 1) {
            if (l & 1) L = fn(L, tr[l++]);
            if (~r & 1) R = fn(tr[r--], R);
        }
        return fn(L, R);
    }
    int find_kth(Node k) {
        if (tr[1] < k) return -1;
        int nd = 1;
        while (nd < siz) {
            nd <= 1;
            if (tr[nd] < k) { k -= tr[nd]; nd |= 1; }
        }
        return nd - siz;
    }
    int find(auto chk) {
        if (!chk(e, tr[1])) return -1;
        int cur = 1; Node pref = e;
        while (cur < siz) {
            if (chk(pref, tr[cur << 1])) cur = cur << 1;
```



```

    else {
        pref = fn(pref, tr[cur << 1]);
        cur = cur << 1 | 1;
    }
}
return cur - siz;
}
};

int main() {
    vector<int> v = {1, 2, 3, 4, 5};
    // 1. RSQ (Range Sum Query)
    SegTree<int> rsq(v, 0, [(int a, int b) { return a+b; }]);
    rsq.add(1, 5); // v[1] += 5
    int sum = rsq.query(1, 3); // Sum of v[1..3]
    int kth = rsq.find_kth(4); // First idx with prefix sum >= 4
    // 2. RMQ (Range Maximum Query)
    SegTree<int> rmq(10, -1e9, [(int a, int b) { return
    max(a, b); }]);
    rmq.update(2, 15); // v[2] = 15
    int mx = rmq.query(0, 5); // Max of v[0..5]
    // 3. SegTree Walk (find), O(log N)
    // pref: left prefix result, nd: current nd result
    int tar = 10;
    int idx = rmq.find([(int pref, int nd) {
        // First idx i where max(v[0..i]) >= tar
        return max(pref, nd) >= tar;
        // First idx i where sum(v[0..i]) >= tar
        // return pref + nd >= tar;
    }]);
}

```

3.3 Inversion counting

```

auto getInvs = [&](vector<ll> v) -> ll {
    if (v.empty()) return 0;
    vector<ll> tmp(sz(v));
    auto solve = [&](auto solve, int l, int r) -> ll {
        if (l >= r) return 0;
        int m = (l+r)/2;
        ll res = solve(solve, l, m) + solve(solve, m+1, r);
        int i = l, j = m+1, k = l;
        while (i <= m && j <= r) {
            if (v[i] <= v[j]) tmp[k++] = v[i++];
            else tmp[k++] = v[j++], res += m-i+1;
        }
        while (i <= m) tmp[k++] = v[i++];
        while (j <= r) tmp[k++] = v[j++];
        for (int i = l; i <= r; i++) v[i] = tmp[i];
        return res;
    };
    return solve(solve, 0, sz(v)-1);
};

```

3.4 1D & 2D Fenwick Tree

Usage: Point updates and subgrid sums on a 2D plane.

Time Complexity: 1D: $O(\log N)$, 2D: $O(\log N \log M)$

```

struct Fenwick {
    const ll MAXN = 100000;
    vector<ll> tree; int SZ;

```

```

    Fenwick(ll sz) : SZ(sz+1), tree(sz+1) {}
    Fenwick() : Fenwick(MAXN) {}
    ll query(ll p) { // sum from index 1 to p, inclusive
        ll ret = 0;
        for (; p > 0; p -= p & -p) ret += tree[p];
        return ret;
    }
    void add(ll p, ll val) {
        for (; p <= SZ; p += p & -p) tree[p] += val;
    }
};

// Fenwick_2d<int> T(n+1,m+1) for nxm grid indexed from 1
template<class T> struct Fenwick_2d {
    vector<vector<T>> x;
    Fenwick_2d(int n, int m) : x(n, vector<T>(m)) {}
    void add(int k1, int k2, int a) { // x[k] += a
        for (; k1 < sz(x); k1 |= k1+1)
            for (int k = k2; k < sz(x[k1]); k |= k+1) x[k1][k] += a;
    }
    T sum(int k1, int k2) { // return x[0]+...+x[k]
        T s = 0;
        for (; k1 >= 0; k1 = (k1 & (k1 + 1)) - 1)
            for (int k = k2; k >= 0; k = (k & (k+1)) - 1) s += x[k1][k];
        return s;
    }
};

```

3.5 Merge Sort Tree

Usage: Count/rank of elements in range $[L, R]$.

Time Complexity: $O(\log^2 N)$ per query

```

template <typename T>
struct MergeSortTree {
    int sz;
    vector<vector<T>> tr; // Space: O(N log N)
    MergeSortTree(int n) {
        sz = 1; while (sz < n) sz <= 1;
        tr.resize(sz*2);
    }
    void add(int x, T v) { tr[x+sz].push_back(v); }
    void build() { // Build: O(N log N)
        for (int i = sz-1; i > 0; i--) {
            tr[i].resize(sz(tr[i*2]) + sz(tr[i*2+1]));
            merge(all(tr[i*2]), all(tr[i*2+1]), tr[i].begin());
        }
    }
    int query(int l, int r, T k) { // Query: O(log^2 N)
        int res = 0;
        for (l += sz, r += sz; l <= r; l >= 1, r >= 1) {
            if (l & 1) {
                res += tr[l].end() - upper_bound(all(tr[l]), k);
                l++;
            }
            if (!(r & 1)) {
                res += tr[r].end() - upper_bound(all(tr[r]), k);
                r--;
            }
            /*
            - Count < k: lower_bound(all(v)) - v.begin()
            - Count <= k: upper_bound(all(v)) - v.begin()
            - Count >= k: v.end() - lower_bound(all(v))
            - Count > k: v.end() - upper_bound(all(v))
            */
        }
    }
};

```

```

    */
}
return res;
}
};

```

3.6 Persistent Segment Tree

Usage: Accessing previous versions and range k-th element.

Time Complexity: $O(\log N)$ per query

```

// PST: 구간 k번째 수 (좌표압축 + 버전 배열)
// 1) 값들을 좌표압축 -> 최대값 = 압축 인덱스
// 2) root[0]=new PSTNode; init(root[0], 0, maxC);
// 3) for i=1..n: root[i]=new PSTNode; update(root[i-1],
root[i], 0, maxC, a[i]);
// 4) kth(root[l-1], root[r], 0, maxC, k) -> 구간 [l,r]의
k번째 작은 값(압축idx)
// root[i] = prefix [1..i] 버전. [l,r] = root[r] - root[l-1]
struct PSTNode{
    PSTNode *l, *r; int v;
    PSTNode(){ l = r = nullptr; v = 0; }
};
PSTNode *root[101010];
PST({} memset(root, 0, sizeof root); } // constructor
void init(PSTNode *node, int s, int e){
    if(s == e) return;
    int m = s + e >> 1;
    node->l = new PSTNode; node->r = new PSTNode;
    init(node->l, s, m); init(node->r, m+1, e);
}
void update(PSTNode *prv, PSTNode *now, int s, int e, int x){
    if (s == e) { now->v = prv ? prv->v + 1 : 1; return; }
    int m = s + e >> 1;
    if (x <= m) {
        now->l = new PSTNode; now->r = prv->r;
        update(prv->l, now->l, s, m, x);
    }
    else {
        now->r = new PSTNode; now->l = prv->l;
        update(prv->r, now->r, m+1, e, x);
    }
    int t1 = now->l ? now->l->v : 0;
    int t2 = now->r ? now->r->v : 0;
    now->v = t1 + t2;
}
int kth(PSTNode *prv, PSTNode *now, int s, int e, int k){
    if (s == e) return s;
    int m = s + e >> 1, diff = now->l->v - prv->l->v;
    if (k <= diff) return kth(prv->l, now->l, s, m, k);
    else return kth(prv->r, now->r, m+1, e, k-diff);
}

```

3.7 Link-Cut Tree

Usage: Dynamic tree structure supporting link, cut, and path operations using auxiliary Splay trees.

Time Complexity: Amortized $O(\log N)$

```
// MAXN은 (초기 크기 + insert 총 횟수) 이상
const ll INF = 1e18, MAXN = 3e5+5;
struct BBST {
    struct Node {
        int l = 0, r = 0, p = 0;
        ll cnt = 0, val = 0, sum = 0, mx = -INF, mn = INF, lz = 0;
        bool flip = false;
    } tr[MAXN];
    int rt = 0, ptr = 0, pos[MAXN];
    void clear() {
        fill(tr, tr+ptr+1, Node{});
        rt = ptr = 0;
    }
    int new_nd(ll v = 0, int id = -1) {
        auto &n = tr[++ptr];
        n.val = n.mx = n.mn = n.sum = v; n.cnt = 1;
        n.l = n.r = n.p = n.lz = 0; n.flip = false;
        if (id != -1) pos[id] = ptr;
        return ptr;
    }
    bool is_rt(int x) { return !tr[x].p || (tr[tr[x].p].l != x
    && tr[tr[x].p].r != x); }
    void update(int x) {
        if (!x) return;
        auto &n = tr[x], &l = tr[n.l], &r = tr[n.r];
        n.cnt = 1+l.cnt+r.cnt; n.sum = n.val + l.sum + r.sum;
        n.mx = max({n.val, n.l ? l.mx : -INF, n.r ? r.mx : -INF});
        n.mn = min({n.val, n.l ? l.mn : INF, n.r ? r.mn : INF});
    }
    void apply(int x, ll v) {
        if (!x) return;
        auto &n = tr[x]; n.val += v; n.lz += v;
        n.mx += v; n.mn += v; n.sum += v*n.cnt;
    }
    void push(int x) {
        if (!x) return;
        auto &n = tr[x];
        if (n.lz) {
            apply(n.l, n.lz); apply(n.r, n.lz); n.lz = 0;
        } if (n.flip) {
            swap(n.l, n.r);
            if (n.l) tr[n.l].flip ^= 1;
            if (n.r) tr[n.r].flip ^= 1;
            n.flip = false;
        }
    }
    void push_all(int x) {
        if (!is_rt(x)) push_all(tr[x].p);
        push(x);
    }
    void rotate(int x) {
        int p = tr[x].p, g = tr[p].p, b;
        if (x == tr[p].l) tr[p].l = b = tr[x].r, tr[x].r = p;
        else tr[p].r = b = tr[x].l, tr[x].l = p;
        tr[x].p = g; tr[p].p = x;
        if (b) tr[b].p = p;
        if (!is_rt(p)) (tr[g].l == p ? tr[g].l : tr[g].r) = x;
        else rt = x;
        update(p); update(x);
    }
}
```

```

}
void splay(int x, int tgt = 0) {
    push_all(x);
    while (tr[x].p != tgt && !is_rt(x)) {
        int p = tr[x].p, g = tr[p].p;
        if (g != tgt && !is_rt(p)) {
            if ((tr[p].l == x) == (tr[g].l == p)) rotate(p);
            else rotate(x);
        } rotate(x);
    }
    if (!tgt && is_rt(x)) rt = x;
}
// k번째 원소를 tg(기본 0=루트) 아래로 splay
void find_kth(int k, int tg = 0) {
    int cur = rt;
    while (push(cur), 1) {
        int lc = tr[tr[cur].l].cnt;
        if (k == lc+1) break;
        if (k <= lc) cur = tr[cur].l;
        else k -= lc+1, cur = tr[cur].r;
    }
    splay(cur, tg);
}
// 구간을 서브트리로 모음 (루트 반환, 1 <= l <= r <= N)
int gather(int l, int r) {
    find_kth(r+2); find_kth(l, rt);
    return tr[tr[rt].l].r;
}
int build(int l, int r, int p, const vector<ll> &v) {
    if (l > r) return 0;
    int mid = (l+r)/2, val = 0, id = -1;
    if (mid > 1 && mid < sz(v)+2) val = v[mid-2], id = mid-1;
    int x = new_nd(val, id);
    tr[x].p = p;
    tr[x].l = build(l, mid-1, x, v);
    tr[x].r = build(mid+1, r, x, v);
    update(x); return x;
}
void init(const vector<ll> &v) { // v는 0-idx vector
    clear();
    rt = build(1, sz(v)+2, 0, v);
}
void add_val(int l, int r, ll v) { apply(gather(l, r), v); }
int get_idx(int id) { // ID 원소의 인덱스 반환 (1-idx)
    push_all(pos[id]); splay(pos[id]);
    return tr[tr[rt].l].cnt;
}
ll get_val(int k) { find_kth(k+1); return tr[rt].val; }
void reverse(int l, int r) { tr[gather(l, r)].flip ^= 1; }
// k번째 뒤(0: 앞 / N: 뒤)에 값 v (ID: id) 삽입
void insert(int k, ll v, int id = -1) {
    find_kth(k+1); find_kth(k+2, rt);
    int x = new_nd(v, id), p = tr[rt].r;
    tr[p].l = x; tr[x].p = p;
    update(p); update(rt); splay(x);
}
void erase(int l, int r) {
    int p = tr[gather(l, r)].p;
    tr[p].r = 0; update(p); splay(p);
}
}
```

```

}
void shift(int l, int r, int k) { // k칸 우측 시프트
    int l = r-l+1; k = (k%L+L) % L;
    if (!k) return;
    reverse(l, r); reverse(l, l+k-1); reverse(l+k, r);
}
int build_val(int l, int r, int p, const vector<pair<ll,
int>> &v) {
    if (l > r) return 0;
    int mid = (l + r) / 2;
    ll val = 0; int id = -1;
    if (mid == 1) val = -INF;
    else if (mid == sz(v) + 2) val = INF;
    else val = v[mid - 2].first, id = v[mid - 2].second;
    int x = new_nd(val, id);
    tr[x].p = p;
    tr[x].l = build_val(l, mid - 1, x, v);
    tr[x].r = build_val(mid + 1, r, x, v);
    update(x); return x;
}
// (val, id) 사전순 정렬된 v(0-idx)로 트리 생성
void init_val(const vector<pair<ll, int>> &v = {}) {
    clear();
    rt = build_val(1, sz(v) + 2, 0, v);
}
// (val, id) >= (s, id) 인 첫 노드를 tg 아래로 splay
int lower_bound(ll s, int id = -1, int tg = 0) {
    int cur = rt, ret = 0;
    while (cur) { push(cur);
        if (pair(tr[cur].val, tr[cur].id) >= pair(s, id)) {
            ret = cur; cur = tr[cur].l;
        }
        else cur = tr[cur].r;
    }
    if (ret) splay(ret, tg);
    return ret;
}
// (val, id) < (s, id) 인 마지막 노드를 tg 아래로 splay
int prev_bound(ll s, int id = -1, int tg = 0) {
    int cur = rt, ret = 0;
    while (cur) { push(cur);
        if (pair(tr[cur].val, tr[cur].id) < pair(s, id)) {
            ret = cur; cur = tr[cur].r;
        }
        else cur = tr[cur].l;
    }
    if (ret) splay(ret, tg);
    return ret;
}
// 값 [(s1, id1), (s2, id2)] 구간을 모음 (루트 반환)
int gather_val(ll s1, int id1, ll s2, int id2) {
    int r = lower_bound(s2, id2), l = prev_bound(s1, id1, r);
    return tr[l].r;
}
// (v, id) 값 노드 정렬 위치에 삽입
void insert_val(ll v, int id = -1) {
    int r = lower_bound(v, id), l = prev_bound(v, id, r);
    int x = new_nd(v, id); tr[l].r = x; tr[x].p = l;
}
```

```

    update(l); update(r); splay(x);
}
void erase_val(ll v, int id = -1) { // (v, id)값 노드 제거
    int r = lower_bound(v, id+1), l = prev_bound(v, id, r);
    tr[l].r = 0; update(l); update(r); splay(l);
}
// === Link-Cut tree ===
// 루트~x 경로를 단일 Splay로 연결
void access(int x) {
    for (int y = 0; x; y = x, x = tr[x].p) {
        splay(x); tr[x].r = y; update(x);
    }
}
void make_rt(int x) { access(x); splay(x); tr[x].flip ^= 1; }
int find_rt(int x) {
    access(x); splay(x);
    while (push(x), tr[x].l) x = tr[x].l;
    splay(x); return x;
}
bool conn(int u, int v) { return find_rt(u) == find_rt(v); }
// link/cur: 간선 (u, v) 추가/제거 (성공 시 true)
bool link(int u, int v) {
    if (conn(u, v)) return false;
    make_rt(u); tr[u].p = v; return true;
}
bool cut(int u, int v) {
    make_rt(u);
    if (find_rt(v) == u && tr[v].p == u && !tr[v].l) {
        tr[v].p = tr[u].r = 0;
        update(u); return true;
    }
    return false;
}
void cut(int x) { // x와 부모 노드 간선 제거
    access(x); splay(x);
    if (tr[x].l) {
        tr[tr[x].l].p = 0; tr[x].l = 0; update(x);
    }
}
int lca(int u, int v) {
    if (!conn(u, v)) return 0;
    access(u); access(v); splay(u);
    return tr[u].p ? tr[u].p : u;
}
// path_xxx: u, v가 연결되어 있을 때만 호출 가능
Node path_qry(int u, int v) {
    make_rt(u); access(v); splay(v);
    return tr[v];
}
void path_upd(int u, int v, ll val) {
    make_rt(u); access(v); splay(v);
    apply(v, val);
}
}
};

```

3.8 Li Chao Tree

```

struct LiChaoTree {
    // min LiChao, 정수, 사용하기 전에 init(min_x, max_x) 호출

```

```

// update(0, {a, b}) : 직선 y=ax+b 추가
// update(0, {a, b}, l, r) : y=ax+b를 x in [l, r]에 추가
// query(0, k) : x=k에서 최솟값 리턴, 직선 없으면 INF
// 쿼리, 직선 update : O(log n), 선분 update : O(log^2 n)
struct Line {
    ll a, b;
    ll get(ll x) { return a*x+b; }
};
struct Node {
    int l, r; ll s, e; Line line;
};
const ll INF = 4e18;
const Line id = {0, INF};
vector<Node> tree;
void init(ll s, ll e) {
    tree.push_back({-1, -1, s, e, id});
}
void update(int node, Line v) {
    ll s = tree[node].s, e = tree[node].e, m = s+(e-s)/2;
    Line low = tree[node].line, high = v;
    if (low.get(s) > high.get(s)) swap(low, high);
    if (low.get(e) <= high.get(e)) {
        tree[node].line = low; return;
    }
    if (low.get(m) < high.get(m)) {
        tree[node].line = low;
        if (tree[node].r == -1) {
            tree[node].r = tree.size();
            tree.push_back({-1, -1, m+1, e, id});
        }
        update(tree[node].r, high);
    } else {
        tree[node].line = high;
        if (tree[node].l == -1) {
            tree[node].l = tree.size();
            tree.push_back({-1, -1, s, m, id});
        }
        update(tree[node].l, low);
    }
}
void update(int node, Line v, ll l, ll r) {
    if (r < tree[node].s || l > tree[node].e) return;
    if (l <= tree[node].s && tree[node].e <= r) {
        update(node, v); return;
    }
    ll s = tree[node].s, e = tree[node].e;
    ll m = s+(e-s)/2;
    if (tree[node].l == -1) {
        tree[node].l = tree.size();
        tree.push_back({-1, -1, s, m, id});
    }
    if (tree[node].r == -1) {
        tree[node].r = tree.size();
        tree.push_back({-1, -1, m+1, e, id});
    }
    update(tree[node].l, v, l, r);
    update(tree[node].r, v, l, r);
}
ll query(int node, ll x){

```

```

    if (node == -1) return INF;
    ll s = tree[node].s, e = tree[node].e;
    ll m = s + (e - s) / 2;
    if (x <= m) return min(tree[node].line.get(x),
        query(tree[node].l, x));
    else return min(tree[node].line.get(x),
        query(tree[node].r, x));
    }
};

```

3.9 DSU (Weighted)

```

struct DSU {
    vector<int> p; int comp;
    DSU(int n) : p(n+1, -1), comp(n) {}
    int find(int x) {
        return p[x] < 0 ? x : p[x] = find(p[x]);
    }
    bool merge(int x, int y) {
        x = find(x); y = find(y);
        if (x == y) return false;
        if (p[x] > p[y]) swap(x, y);
        p[x] += p[y]; p[y] = x; comp--;
        return true;
    }
    bool same(int x, int y) { return find(x) == find(y); }
    int size(int x) { return -p[find(x)]; }
};
struct WeightedDSU {
    vector<int> p, d;
    WeightedDSU(int n) : p(n+1), d(n+1, 0) { iota(all(p), 0); }
    int find(int x) {
        if (p[x] == x) return x;
        int r = find(p[x]); d[x] += d[p[x]];
        return p[x] = r;
    }
    int diff(int a, int b) { find(a); find(b); return
        d[a]-d[b]; } // for parity: ret &1
    bool merge(int a, int b, int w) { // a-b = w
        int pa = find(a), pb = find(b);
        if (pa == pb) return diff(a, b) == w;
        d[pa] = w + d[b]-d[a]; p[pa] = pb;
        return true;
    }
    bool same(int a, int b) { return find(a) == find(b); }
};

```

3.10 Trie

```

// Trie<10,0> tr;
template <int S = 26, char OS = 'a'> struct Trie {
    struct Node {
        int nxt[S], pass = 0, end = 0;
        Node() { memset(nxt, -1, sizeof(nxt)); }
    };
    vector<Node> tree;
    Trie() { init(); }
    void init() { tree.clear(); tree.emplace_back(); }
    // 원소 추가(1)/삭제(-1), 0(|s|)

```



```

void insert(const auto &s, int c = 1) {
    int u = 0; tree[u].pass += c;
    for (auto i : s) {
        int idx = i - 0S;
        if (tree[u].nxt[idx] == -1) {
            tree[u].nxt[idx] = sz(tree); tree.emplace_back();
        }
        u = tree[u].nxt[idx]; tree[u].pass += c;
    }
    tree[u].end += c;
}
// 원소 삭제, O(|s|)
bool erase(const auto& s, int c = 1) {
    if (count(s) < c) return false;
    insert(s, -c); return true;
}
// 완전 일치 개수, O(|s|)
int count(const auto& s) const {
    int u = 0;
    for (auto i : s) {
        int idx = i - 0S;
        if (u == -1 || tree[u].nxt[idx] == -1) return 0;
        u = tree[u].nxt[idx];
    }
    return tree[u].end;
}
// prefix로 포함 개수, O(|s|)
int count_prefix(const auto& s) const {
    int u = 0;
    for (auto i : s) {
        int idx = i - 0S;
        if (u == -1 || tree[u].nxt[idx] == -1) return 0;
        u = tree[u].nxt[idx];
    }
    return tree[u].pass;
}
// 사전순 k번째 문자열, O(|s| * S)
string kth(int k) const {
    if (k <= 0 || k > tree[0].pass) return "";
    int u = 0; string res = "";
    while (true) {
        if (k <= tree[u].end) return res;
        k -= tree[u].end;
        for (int i = 0; i < S; i++) {
            int v = tree[u].nxt[i];
            if (v != -1 && tree[v].pass > 0) {
                if (k <= tree[v].pass) {
                    res += (char)(i + 0S);
                    u = v; break;
                }
                k -= tree[v].pass;
            }
        }
    }
}
};
template <typename T = ll, int B = 62> struct XorTrie {
    struct Node {
        int nxt[2]{ -1, -1 }, cnt = 0; // 서브트리 원소개수
    };
    vector<Node> tree;
    XorTrie() { init(); }
    void init() { tree.clear(); tree.emplace_back(); }
    // 원소 추가(1)/삭제(-1), O(B)
    void insert(T x, int c = 1) {
        int u = 0; tree[u].cnt += c;
        for (int i = B-1; i >= 0; i--) {
            int b = (x >> i) & 1;
            if (tree[u].nxt[b] == -1) {
                tree[u].nxt[b] = sz(tree); tree.emplace_back();
            }
            u = tree[u].nxt[b]; tree[u].cnt += c;
        }
    }
    // 원소 삭제, O(B)
    bool erase(T x, int c = 1) {
        if (count(x) < c) return false;
        insert(x, -c); return true;
    }
    // x 개수 조회, O(B)
    int count(T x) const {
        int u = 0;
        for (int i = B-1; i >= 0; i--) {
            int b = (x >> i) & 1;
            if (u == -1 || tree[u].nxt[b] == -1) return 0;
            u = tree[u].nxt[b];
        }
        return tree[u].cnt;
    }
    // max(x ^ y), O(B)
    T max_xor(T x) const {
        int u = 0; T res = 0;
        for (int i = B-1; i >= 0; i--) {
            int b = (x >> i) & 1, p = b ^ 1;
            if (tree[u].nxt[p] != -1 && tree[tree[u].nxt[p]].cnt > 0) {
                res |= (T(1) << i); u = tree[u].nxt[p];
            } else {
                u = tree[u].nxt[b];
            }
        }
        return res;
    }
    // min(x ^ y), O(B)
    T min_xor(T x) const {
        int u = 0; T res = 0;
        for (int i = B-1; i >= 0; i--) {
            int b = (x >> i) & 1;
            if (tree[u].nxt[b] != -1 && tree[tree[u].nxt[b]].cnt > 0) {
                u = tree[u].nxt[b];
            } else {
                res |= (T(1) << i); u = tree[u].nxt[b ^ 1];
            }
        }
        return res;
    }
    // x^y 기준 k번째 최솟값(1-idx), O(B)
};

```

```

T kth_xor(T x, int k) const {
    if (k <= 0 || k > tree[0].cnt) return -1;
    int u = 0; T res = 0;
    for (int i = B-1; i >= 0; i--) {
        int b = (x >> i) & 1, l = tree[u].nxt[b], l_cnt = (l
        != -1 ? tree[l].cnt : 0);
        if (k <= l_cnt) {
            u = l;
        } else {
            k -= l_cnt; res |= (T(1) << i);
            u = tree[u].nxt[b ^ 1];
        }
    }
    return res;
}
// (x^y) < k 만족하는 y 개수, O(B)
int count_less(T x, T k) const {
    int u = 0, res = 0;
    for (int i = B-1; i >= 0; i--) {
        if (u == -1) break;
        int bx = (x >> i) & 1, bk = (k >> i) & 1;
        if (bk) {
            int same = tree[u].nxt[bx];
            if (same != -1) res += tree[same].cnt;
            u = tree[u].nxt[bx ^ 1];
        } else {
            u = tree[u].nxt[bx];
        }
    }
    return res;
}
};

```

3.11 Sparse Table

```

template <typename T, typename Op>
struct SparseTable {
    int n, lg; Op f;
    vector<vector<T>> st;
    SparseTable(const vector<T>& v, Op op) : n(sz(v)), f(op) {
        lg = __lg(max(1, n)) + 1;
        st.assign(lg, vector<T>(n)); st[0] = v;
        for (int i = 1; i < lg; i++) {
            int len = 1 << i;
            for (int m = len; m < n+len; m += len*2) {
                st[i][m-1] = v[m-1];
                for (int j = m-2; j >= max(0, m-len); j--)
                    st[i][j] = f(v[j], st[i][j+1]);
                if (m < n) {
                    st[i][m] = v[m];
                    for (int j = m+1; j < min(n, m+len); j++)
                        st[i][j] = f(st[i][j-1], v[j]);
                }
            }
        }
    }
    T query(int l, int r) const { // [l, r], 0-idx
        if (l == r) return st[0][l];
        int k = __lg(r-l+1);
    }
};

```

```

    return f(st[k][l], st[k][r]);
}
};
// st(v, [(l a, l b){ return min(a, b); }]); // RMQ
// st(v, [(l a, l b){ return gcd(a, b); }]); // Range GCD
// st(v, [(l a, l b){ return a + b; }]); // Range Sum

```

3.12 Mo's Algorithm

```

struct Query { int l, r, idx; };
vector<ll> mos(vector<int> a, vector<Query> q) {
    int n = sz(a);
    auto tmp = a; compress(tmp);
    for (auto &x : a) x = lower_bound(all(tmp), x) -
tmp.begin();
    int k = *max_element(all(a)) + 1;
    int B = max(1, (int)(n / sqrt(max(1, sz(q)))));
    sort(all(q), [&](const Query& x, const Query& y) {
        int bx = x.l / B, by = y.l / B;
        if (bx != by) return bx < by;
        return (bx & 1) ? x.r > y.r : x.r < y.r;
    });
    vector<ll> cnt(k), res(sz(q));
    ll now = 0;
    auto add = [&](int i) { if (cnt[a[i]]++ == 0) now++; };
    auto remove = [&](int i) { if (--cnt[a[i]] == 0) now--; };
    int l = 0, r = 0; // 0-idx, [l, r)
    for (auto &[ql, qr, idx] : q) {
        while (l > ql) add(--l);
        while (r < qr) add(r++);
        while (l < ql) remove(l++);
        while (r > qr) remove(--r);
        res[idx] = now;
    }
    return res;
}

```

4 Graph

4.1 Bellman Ford

Usage: SSSP with negative weights/cycles.

Time Complexity: $O(VE)$

```

auto bellman = [&](int s) -> bool {
    fill(all(d), INF); d[s] = 0; bool chk = 0;
    for (int i = 0; i < n; i++) { chk = 0;
        for (int u = 1; u <= n; u++) {
            if (d[u] == INF) continue;
            for (auto [w, v] : adj[u]) if (d[v] > d[u] + w) {
                d[v] = d[u] + w; chk = 1; if (i == n - 1) return 0;
            }
        }
        if (!chk) break;
    }
    return 1;
};

```

4.2 LCA

Usage: Lowest Common Ancestor using binary lifting.

Time Complexity: $O(\log N)$

```

int N, Q, D[101010], P[22][101010];
vector<int> G[101010];
void Connect(int u, int v){
    G[u].push_back(v); G[v].push_back(u);
}
void DFS(int v, int b=-1){
    for(auto i : G[v]) if(i != b) D[i] = D[v] + 1, P[0][i] =
v, DFS(i, v);
}
int LCA(int u, int v){
    if(D[u] < D[v]) swap(u, v);
    int diff = D[u] - D[v];
    for(int i=0; diff; i++, diff>>=1) if(diff & 1) u = P[i][u];
    if(u == v) return u;
    for(int i=21; i>=0; i--) if(P[i][u] != P[i][v]) u =
P[i][u], v = P[i][v];
    return P[0][u];
}
// 1. Connect로 간선 추가 2. DFS(1) 호출 3. 아래코드 실행
for(int i=1; i<22; i++) for(int j=1; j<=N; j++) P[i][j] =
P[i-1][P[i-1][j]];
// 4. LCA(u, v)로 최소 공통 조상 구할 수 있음

```

4.3 HLD

Usage: Heavy-Light Decomposition for path queries on trees.

Time Complexity: $O(\log^2 N)$

// HLD: 트리 경로/서브트리 쿼리를 세그트리 구간으로 변환
(1-idx 정점).
// hld.build(a) // a[u]=정점 u 값 (1-indexed). 정점/간선
가중치 모두 지원
// path_upd(u,v,x, edge=0) // 경로 u~v에 x 적용 (edge=1이면
LCA 정점 제외, 간선용)
// setv(u,x)/addv(u,x) // 정점 단일 갱신
// qsum/qmin/qmax(u,v, edge=0) // 경로 쿼리
// * edge=true: 간선 가중치 문제에서 씬 (정점을 간선에 대응,
LCA 빼고)

```

template<typename T> struct SegTree {
    struct Nd { T sum, mn, mx; };
    struct Lz { T add, set; bool has; };
    int n; vector<Nd> tr; vector<Lz> lz;
    SegTree(int n) : n(n), tr(4*n+1), lz(4*n+1, { 0, 0, 0 }) {}
    Nd id() { return { 0, numeric_limits<T>::max(),
numeric_limits<T>::lowest() }; }
    bool empty(Lz f) { return !f.has && f.add==0; }
    Nd merge(Nd a, Nd b) { return { a.sum+b.sum,
min(a.mn,b.mn), max(a.mx,b.mx) }; }
    Lz comp(Lz f, Lz g) { // comp(f, g)(x) = f(g(x))
        if (f.has) return f;
        g.add += f.add;
        return g;
    }
    void apply(int nd, int s, int e, Lz f) {
        int len = e-s+1;
        if (f.has) tr[nd] = { f.set * len, f.set, f.set };
        tr[nd].sum += f.add * len;
        tr[nd].mn += f.add; tr[nd].mx += f.add;
        lz[nd] = comp(f, lz[nd]);
    }
}

```

```

void push(int nd, int s, int e) {
    if (empty(lz[nd])) return;
    if (s != e) {
        int m = (s + e) >> 1;
        apply(nd<<1, s, m, lz[nd]);
        apply(nd<<1|1, m+1, e, lz[nd]);
    }
    lz[nd] = { 0, 0, 0 };
}
void build(int nd, int s, int e, const vector<T>& a) {
    if (s == e) return void(tr[nd] = { a[s], a[s], a[s] });
    int m = (s+e)>>1;
    build(nd<<1, s, m, a);
    build(nd<<1|1, m+1, e, a);
    tr[nd] = merge(tr[nd<<1], tr[nd<<1|1]);
}
void upd(int nd, int s, int e, int l, int r, Lz f) {
    if (l > r || r < s || e < l) return;
    push(nd, s, e);
    if (l <= s && e <= r) {
        apply(nd, s, e, f);
        return;
    }
    int m = (s+e)>>1;
    upd(nd<<1, s, m, l, r, f);
    upd(nd<<1|1, m+1, e, l, r, f);
    tr[nd] = merge(tr[nd<<1], tr[nd<<1|1]);
}
Nd qry(int nd, int s, int e, int l, int r) {
    push(nd, s, e);
    if (l > r || r < s || e < l) return id();
    if (l <= s && e <= r) return tr[nd];
    int m = (s+e)>>1;
    return merge(qry(nd<<1, s, m, l, r), qry(nd<<1|1, m+1,
e, l, r));
}
void build(const vector<T>& a) { build(1, 0, n-1, a); }
void add(int l, int r, T x) { upd(1, 0, n-1, l, r, { x, 0,
0 }); }
void set(int l, int r, T x) { upd(1, 0, n-1, l, r, { 0, x,
1 }); }
Nd query(int l, int r) { return qry(1, 0, n-1, l, r); }
T qsum(int l, int r) { return qry(1, 0, n-1, l, r).sum; }
T qmin(int l, int r) { return qry(1, 0, n-1, l, r).mn; }
T qmax(int l, int r) { return qry(1, 0, n-1, l, r).mx; }
};
template<typename T = ll> struct HLD {
    int n, pv;
    vector<vector<int>> adj;
    vector<int> siz, dep, par, top, in;
    SegTree<T> seg;
    HLD(int n) : n(n), adj(n+1), siz(n+1), dep(n+1), par(n+1),
top(n+1), in(n+1), seg(n) {}
    void add(int u, int v) { adj[u].push_back(v); }
    void dfs1(int u, int p) {
        siz[u] = 1; dep[u] = dep[p]+1; par[u] = p;
        for (auto& v : adj[u]) {

```

```

    if (v == p) continue;
    dfs1(v, u); siz[u] += siz[v];
    if (adj[u][0] == p || siz[v] > siz[adj[u][0]]) swap(v,
    adj[u][0]);
}
}
void dfs2(int u, int p) {
    in[u] = pv++;
    for (auto v : adj[u]) {
        if (v == p) continue;
        top[v] = (v == adj[u][0] ? top[u] : v);
        dfs2(v, u);
    }
}
void build(const vector<T>& a = {}) { // a[u]: vertex
    value, 1-indexed
    pv = 0;
    for (int i = 1; i <= n; i++) {
        if (!siz[i]) {
            top[i] = i;
            dfs1(i, 0);
            dfs2(i, 0);
        }
    }
    vector<T> base(n);
    if (sz(a) for (int i = 1; i <= n; i++) base[in[i]] =
    a[i];
    seg.build(base);
}
int lca(int u, int v) {
    while (top[u] != top[v]) {
        if (dep[top[u]] < dep[top[v]]) swap(u, v);
        u = par[top[u]];
    }
    return dep[u] < dep[v] ? u : v;
}
int path_len(int u, int v) {
    return dep[u] + dep[v] - 2*dep[lca(u, v)] + 1;
}
void path_upd(int u, int v, T x, bool edge = false) {
    while (top[u] != top[v]) {
        if (dep[top[u]] < dep[top[v]]) swap(u, v);
        seg.add(in[top[u]], in[u], x); // or seg.set
        u = par[top[u]];
    }
    if (dep[u] > dep[v]) swap(u, v);
    if (in[u] + edge <= in[v]) {
        seg.add(in[u] + edge, in[v], x); // or seg.set
    }
}
void setv(int u, T x) { seg.set(in[u], in[u], x); }
void addv(int u, T x) { seg.add(in[u], in[u], x); }
typename SegTree<T>::Nd path_qry(int u, int v, bool edge =
false) {
    typename SegTree<T>::Nd res = seg.id();
    while (top[u] != top[v]) {
        if (dep[top[u]] < dep[top[v]]) swap(u, v);
        res = seg.merge(res, seg.query(in[top[u]], in[u]));
        u = par[top[u]];
    }
}

```

```

    }
    if (dep[u] > dep[v]) swap(u, v);
    return seg.merge(res, seg.query(in[u]+edge, in[v]));
}
T qsum(int u, int v, bool edge = false) { return
path_qry(u, v, edge).sum; }
T qmin(int u, int v, bool edge = false) { return
path_qry(u, v, edge).mn; }
T qmax(int u, int v, bool edge = false) { return
path_qry(u, v, edge).mx; }
};

```

4.4 Centroid Decomposition

Usage: Divide and conquer on trees for path/distance problems.

Time Complexity: $O(N \log N)$ build

```

struct CentroidTree {
    vector<vector<int>> adj, c_adj; // adj: 원본트리 / c_adj:
    센트로이드트리
    vector<int> sz, par, vis; int N;
    CentroidTree(int n) : N(n), adj(n+1), c_adj(n+1), sz(n+1),
    par(n+1), vis(n+1) {}
    void add_edge(int u, int v) { adj[u].push_back(v);
    adj[v].push_back(u); }
    int get_sz(int curr, int prev) {
        sz[curr] = 1;
        for (int next : adj[curr])
            if (next != prev && !vis[next]) sz[curr] +=
            get_sz(next, curr);
        return sz[curr];
    }
    int get_cent(int curr, int prev, int to_sz) {
        for (int next : adj[curr])
            if (next != prev && !vis[next] && sz[next] > to_sz / 2)
                return get_cent(next, curr, to_sz);
        return curr;
    }
    void solve(int u) {
        // * solve(u): 현재 centroid u를 지나는 모든 경로를 여기서
        계산.
        // - u에서 각 서브트리로 DFS하며 거리/조건 집계
        // - 서로 다른 서브트리에서 오는 경로끼리 쌍을 만들 때
        중복 주의
        // build(1) 호출하면 자동으로 centroid tree 완성.
    }
    int build(int curr, int p = -1) {
        int cent = get_cent(curr, -1, get_sz(curr, -1));
        solve(cent);
        vis[cent] = 1; par[cent] = p;
        for (int next : adj[cent]) {
            if (!vis[next]) {
                int child = build(next, cent);
                c_adj[cent].push_back(child); // 센트로이드 계층 연결
            }
        }
        return cent;
    }
};

```

4.5 Bipartite Matching

Usage: Maximum matching in bipartite graphs.

Time Complexity: $O(E\sqrt{V})$

```

struct BiMatch { // Hopcroft-Karp
    vector<vector<int>> graph, grev;
    vector<int> mA, mB, dist, work;
    vector<bool> visA, visB, fA, fB; // vertex i can be
    excluded from some max matching
    int ns, ms;
    BiMatch(int n, int m) : ns(n), ms(m), graph(n+1),
    grev(m+1), mA(n+1), mB(m+1), dist(n+1), work(n+1) {}
    void add(int a, int b) { graph[a].push_back(b);
    grev[b].push_back(a); }
    void bfs() {
        fill(all(dist), -1);
        queue<int> q;
        for (int i = 1; i <= ns; i++) if (!mA[i]) {
            dist[i] = 0; q.push(i);
        }
        while (!q.empty()) {
            int i = q.front(); q.pop();
            for (auto j : graph[i]) {
                int k = mB[j];
                if (k && dist[k] == -1) {
                    dist[k] = dist[i] + 1; q.push(k);
                }
            }
        }
    }
    bool dfs(int cur) {
        for (int& i = work[cur]; i < sz(graph[cur]); i++) {
            int nb = graph[cur][i], ori = mB[nb];
            if (!ori || dist[ori] == dist[cur] + 1 && dfs(ori)) {
                mA[cur] = nb; mB[nb] = cur; return true;
            }
        }
        return false;
    }
    int match() {
        int ans = 0;
        for (int i = 1; i <= ns; i++) for (int j : graph[i]) {
            if (!mB[j]) {
                mA[i] = j; mB[j] = i;
                ans++; break;
            }
        }
        while (1) {
            fill(all(work), 0); bfs();
            int cnt = 0;
            for (int i = 1; i <= ns; i++) {
                if (!mA[i] && dfs(i)) cnt++;
            }
            if (!cnt) break;
            ans += cnt;
        }
        return ans;
    }
};

```

```

void chkEss() {
    fA.assign(ns+1, 0); fB.assign(ms+1, 0);
    visA.assign(ns+1, 0); visB.assign(ms+1, 0);
    queue<int> q;
    for (int i = 1; i <= ns; i++) if (!mA[i]) {
        fA[i] = visA[i] = 1; q.push(i);
    }
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : graph[u]) if (!visB[v]) {
            visB[v] = 1; int r = mB[v];
            if (r && !fA[r]) {
                fA[r] = visA[r] = 1; q.push(r);
            }
        }
    }
    for (int i = 1; i <= ms; i++) if (!mB[i]) {
        fB[i] = 1; q.push(i);
    }
    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (int u : grev[v]) {
            int r = mA[u];
            if (r && !fB[r]) {
                fB[r] = 1; q.push(r);
            }
        }
    }
}

pair<vector<int>, vector<int>> vertex() { // find minimum
    vertex cover
    chkEss(); vector<int> va, vb;
    for (int i = 1; i <= ns; i++) if (!visA[i])
        va.push_back(i);
    for (int i = 1; i <= ms; i++) if (visB[i])
        vb.push_back(i);
    return { va, vb };
}

}

/* struct BiMatch { // Kuhn's Algorithm
    vector<vector<int>> graph;
    vector<int> mA, mB, vis; int ns, ms;
    BiMatch(int n, int m) : ns(n), ms(m), graph(n+1), mA(n+1),
        mB(m+1), vis(n+1) {}
    void add(int a, int b) { graph[a].push_back(b); }
    bool dfs(int cur) {
        vis[cur] = 1;
        for (int i : graph[cur]) {
            if (mB[i] == 0) {
                mA[cur] = i; mB[i] = cur; return true;
            }
        }
        for (auto i : graph[cur]) {
            int ori = mB[i];
            if (ori == 0 || (!vis[ori] && dfs(ori))) {
                mA[cur] = i; mB[i] = cur; return true;
            }
        }
    }
    return false;
}

```

```

}
int match() {
    int res = 0;
    for (int i = 1; i <= ns; i++) {
        if (mA[i]) continue;
        fill(all(vis), 0);
        if (dfs(i)) res++;
    }
    return res;
}
}; */

4.6 Hungarian
Time Complexity:  $O(N^3)$ 
template<typename T = ll> struct Hungarian {
    vector<T> u, v, ans; vector<int> p, way;
    int n, m; const T INF = 1e15;
    Hungarian(int n, int m) : n(n), m(m), u(n+1), v(m+1),
        p(m+1), way(m+1) {}
    T solve (const vector<vector<T>> &a) {
        ans.clear();
        vector<T> mv(m+1); vector<bool> vis(m+1);
        for (int i = 1; i <= n; i++) {
            fill(all(mv), INF); fill(all(vis), false);
            p[0] = i; int j0 = 0;
            while (p[j0] != 0) {
                vis[j0] = true; T dt = INF;
                int i0 = p[j0], j1 = 0;
                for (int j = 1; j <= m; j++) {
                    if (vis[j]) continue;
                    T cur = a[i0][j] - u[i0] - v[j];
                    if (cur < mv[j]) mv[j] = cur, way[j] = j0;
                    if (mv[j] < dt) dt = mv[j], j1 = j;
                }
                for (int j = 0; j <= m; j++) {
                    if (vis[j]) u[p[j]] += dt, v[j] -= dt;
                    else mv[j] -= dt;
                }
                j0 = j1;
            }
            while (j0 != 0) {
                int j1 = way[j0];
                p[j0] = p[j1]; j0 = j1;
            }
            ans.push_back(-v[0]);
        }
        return sz(ans) ? ans.back() : 0;
    }
    T cost(int k) { return ans[k-1]; }
    vector<int> match() {
        vector<int> res(n+1);
        for (int i = 1; i <= m; i++) if (p[i]) res[p[i]] = i;
        return res;
    }
};

```

4.7 Dinic

Usage: Efficient maximum flow algorithm.

Time Complexity: $O(V^2E)$

```

const ll INF = 1e18;
struct Dinic {
    struct Edge { int to; ll cap; int rev; };
    vector<vector<Edge>> graph;
    vector<int> level, work; int n;
    Dinic(int n) : n(n), graph(n+1), level(n+1), work(n+1) {}
    void add(int u, int v, ll cap) {
        graph[u].push_back({ v, cap, sz(graph[v]) });
        graph[v].push_back({ u, 0, sz(graph[u])-1 });
    }
    bool bfs(int s, int t) {
        fill(all(level), -1); level[s] = 0;
        queue<int> q; q.push(s);
        while (!q.empty()) {
            int cur = q.front(); q.pop();
            for (auto [nxt, cap, rev] : graph[cur]) {
                if (cap > 0 && level[nxt] == -1) {
                    level[nxt] = level[cur]+1;
                    if (nxt == t) return true;
                    q.push(nxt);
                }
            }
        }
        return (level[t] != -1);
    }
    ll dfs(int cur, int t, ll flow) {
        if (cur == t) return flow;
        for (int& i = work[cur]; i < sz(graph[cur]); i++) {
            auto& [nxt, cap, rev] = graph[cur][i];
            if (cap > 0 && level[nxt] == level[cur]+1) {
                ll push = dfs(nxt, t, min(flow, cap));
                if (push > 0) {
                    cap -= push; graph[nxt][rev].cap += push;
                    return push;
                }
            }
        }
        return 0;
    }
    ll flow(int s, int t) {
        ll ans = 0;
        while (bfs(s, t)) {
            fill(all(work), 0);
            while (auto flow = dfs(s, t, INF)) ans += flow;
        }
        return ans;
    }
    vector<bool> mincut(int s) {
        vector<bool> vis(n+1); vis[s] = true;
        queue<int> q; q.push(s);
        while (!q.empty()) {
            int cur = q.front(); q.pop();
            for (auto [nxt, cap, rev] : graph[cur]) {
                if (cap > 0 && !vis[nxt]) {
                    vis[nxt] = true; q.push(nxt);
                }
            }
        }
    }
}

```

```

    }
    return vis;
}
};

```

4.8 MCMF

Usage: Minimum Cost Maximum Flow.(zkw MCMF)

Time Complexity: $O(F \cdot E \log V)$

```

template<typename T = ll>
struct MCMF {
    struct Edge { int to, rev; ll cap; T cost; };
    const T INF = 1e18, EPS = 1e-9;
    vector<vector<Edge>> graph; vector<T> dist;
    vector<int> ptr; vector<bool> vis; int n;
    MCMF(int n) : n(n), graph(n+1), dist(n+1), ptr(n+1),
        vis(n+1) {}
    void add(int u, int v, ll cap, T cost) {
        graph[u].push_back({ v, sz(graph[v]), cap, cost });
        graph[v].push_back({ u, sz(graph[u])-1, 0, -cost });
    }
    bool spfa(int s, int t) {
        fill(all(dist), INF); fill(all(vis), 0);
        deque<int> q; q.push_back(s); dist[s] = 0; vis[s] = 1;
        while (!q.empty()) {
            if (dist[q.front()] > dist[q.back()]) swap(q.front(),
                q.back());
            int cur = q.front(); q.pop_front(); vis[cur] = 0;
            for (auto& [nxt, rev, cap, cost] : graph[cur]) {
                if (cap > 0 && dist[nxt] > dist[cur] + cost + EPS) {
                    dist[nxt] = dist[cur] + cost;
                    if (!vis[nxt]) {
                        vis[nxt] = 1;
                        if (!q.empty() && dist[nxt] < dist[q.front()])
                            q.push_front(nxt);
                        else q.push_back(nxt);
                    }
                }
            }
        }
        return dist[t] < INF;
    }
    ll dfs(int cur, int t, ll flow, T& cost) {
        if (cur == t) return flow;
        vis[cur] = 1;
        for (int& i = ptr[cur]; i < sz(graph[cur]); i++) {
            auto& [nxt, rev, cap, cst] = graph[cur][i];
            if (!vis[nxt] && cap > 0 && abs(dist[nxt] - (dist[cur]
                + cst)) <= EPS) {
                ll push = dfs(nxt, t, min(flow, cap), cost);
                if (push > 0) {
                    graph[nxt][rev].cap -= push;
                    cost += push * cst; cap -= push;
                    vis[cur] = 0; return push;
                }
            }
        }
        return 0;
    }
};

```

```

pair<ll,T> flow(int s, int t) {
    ll res = 0; T cost = 0;
    while (spfa(s, t)) {
        // if (dist[t] >= -EPS) break; // Min-Cost Flow
        fill(all(ptr), 0); fill(all(vis), 0);
        while (1) {
            ll push = dfs(s, t, 1e18, cost);
            if (push == 0) break;
            res += push;
        }
    }
    return { res, cost };
}
};

```

4.9 Circulation

Usage: Flow with lower and upper bounds.

```

const ll INF = 1e18;
struct Dinic {}; // or MCMF
struct Circulation {
    vector<ll> demand, low; vector<pair<int,int>> edge;
    int n, S, T; Dinic dn;
    Circulation(int n) : n(n), S(n+1), T(n+2), demand(n+3, 0),
        dn(n+2) {};
    void add_demand(int u, ll d) { demand[u] += d; }
    int add(int u, int v, ll l, ll r) {
        demand[u] -= l; demand[v] += l;
        dn.add(u, v, r - l); low.push_back(l);
        edge.push_back({ u, sz(dn.graph[u])-1 });
        return sz(edge)-1;
    }
    ll solve() {
        ll sum = 0, res = 0;
        for (int i = 1; i <= n; i++) sum += demand[i];
        if (sum != 0) return false;
        for (int i = 1; i <= n; i++) {
            if (demand[i] > 0) {
                dn.add(S, i, demand[i]); res += demand[i];
            }
            else if (demand[i] < 0) dn.add(i, T, -demand[i]);
        }
        ll f = dn.flow(S, T);
        return (f != res ? -1 : f);
    }
    ll get_flow(int i) { // get actual flow
        auto [u, idx] = edge[i];
        int v = dn.graph[u][idx].to, rev = dn.graph[u][idx].rev;
        return dn.graph[v][rev].cap + low[i];
    }
    // min flow: int t = cc.add(T, S, 0, INF); cc.solve();
    cc.cut(t); cc.dn.flow(N, 1);
    void cut(int i) {
        auto [u, idx] = edge[i];
        auto& fwd = dn.graph[u][idx];
        auto& bwd = dn.graph[fwd.to][fwd.rev];
        fwd.cap = bwd.cap = 0;
    }
};
};

```

4.10 SCC

Usage: Strongly Connected Components.

Time Complexity: $O(V + E)$

```

struct SCC {
    int n, cnt, t; vector<vector<int>> adj;
    vector<int> dfn, low, id, ins; stack<int> st;
    SCC(int n) : n(n), adj(n), dfn(n, -1), low(n, -1), id(n,
        -1), ins(n), cnt(0), t(0) {}
    void add(int u, int v) { adj[u].push_back(v); }
    void dfs(int u) {
        dfn[u] = low[u] = ++t;
        st.push(u); ins[u] = 1;
        for (auto v : adj[u]) {
            if (dfn[v] == -1) {
                dfs(v); low[u] = min(low[u], low[v]);
            }
            else if (ins[v]) {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if (low[u] == dfn[u]) {
            while (true) {
                int v = st.top(); st.pop();
                ins[v] = 0; id[v] = cnt;
                if (u == v) break;
            }
            cnt++;
        }
    }
    void build() {
        for (int i = 0; i < n; i++) if (dfn[i] == -1) dfs(i);
    }
};

```

4.11 2-sat

Usage: Solves 2-SAT in $O(V + E)$; x_i is true if

$SCC(x_i) < SCC(\neg x_i)$.

Time Complexity: $O(V + E)$

```

struct TwoSat { // 0-idx
    int n, t, cnt;
    vector<vector<int>> adj;
    vector<int> dfn, low, scc, st;
    vector<bool> ans;
    TwoSat(int n) : n(n), adj(2*n), dfn(2*n), low(2*n),
        scc(2*n) {}
    int I(int x) { return x^1; }
    void add_edge(int u, int v) { adj[u].push_back(v);
        adj[I(v)].push_back(I(u)); } // u->v
    void add_clause(int u, int v) { add_edge(I(u), v); }
    void add_clause(int u, bool ut, int v, bool vt) {
        add_clause(u*2 + !ut, v*2 + !vt); } // (u==ut) or (v==vt)
    int add() {
        auto New = [&](auto&... vs) { (vs.emplace_back(), ...); };
        for (int i = 0; i < 2; i++) New(adj, dfn, low, scc);
        return n++;
    }
    void most_one(const vector<int> &v, bool cd = true) {
};

```



```

int pre = -1;
for (auto i : v) {
    int x = i*2 + !cd, now = add()*2;
    add_edge(x, now);
    if (pre != -1) { add_edge(pre, now); add_edge(pre,
        I(x)); }
    pre = now;
}
}
void dfs(int u) {
    dfn[u] = low[u] = ++t; st.push_back(u);
    for (auto v : adj[u]) {
        if (dfn[v] == -1) { dfs(v); low[u] = min(low[u],
            low[v]); }
        else if (scc[v] == -1) low[u] = min(low[u], dfn[v]);
    }
    if (low[u] == dfn[u]) {
        while (1) {
            int v = st.back(); st.pop_back();
            scc[v] = cnt;
            if (u == v) break;
        }
        cnt++;
    }
}
bool solve() {
    t = cnt = 0; st.clear();
    fill(all(dfn), -1); fill(all(scc), -1);
    for (int i = 0; i < 2*n; i++) if (dfn[i] == -1) dfs(i);
    ans.resize(n);
    for (int i = 0; i < n; i++) {
        if (scc[i*2] == scc[i*2+1]) return false;
        ans[i] = scc[i*2] < scc[i*2+1];
    }
    return true;
}
};

```

4.12 BCC

Usage: Biconnected Components, Cut-vertices, and Bridges.

Time Complexity: $O(V + E)$

// 1-based, 다른 거 호출하기 전에 tarjan 먼저 호출
 vector<int> G[MAX_V]; int In[MAX_V], Low[MAX_V], P[MAX_V];
 void addEdge(int s, int e) { G[s].push_back(e);
 G[e].push_back(s); }

```

void tarjan(int n) { /// Pre-Process
    int pv = 0;
    function<void(int,int)> dfs = [&pv,&dfs](int v, int b){
        In[v] = Low[v] = ++pv; P[v] = b;
        for(auto i : G[v]){
            if(i == b) continue;
            if(!In[i]) dfs(i, v), Low[v] = min(Low[v], Low[i]);
            else Low[v] = min(Low[v], In[i]);
        }
    };
    for(int i=1; i<=n; i++) if(!In[i]) dfs(i, -1);
}
vector<int> cutVertex(int n){
    vector<int> res; array<char,MAX_V> isCut; isCut.fill(0);

```

```

function<void(int)> dfs = [&dfs,&isCut](int v){
    int ch = 0;
    for(auto i : G[v]){
        if(P[i] != v) continue; dfs(i); ch++;
        if(P[v] == -1 && ch > 1) isCut[v] = 1;
        else if(P[v] != -1 && Low[i] >= In[v]) isCut[v]=1;
    }
};
for(int i=1; i<=n; i++) if(P[i] == -1) dfs(i);
for(int i=1; i<=n; i++) if(isCut[i]) res.push_back(i);
return move(res);
}
vector<PII> cutEdge(int n){
    vector<PII> res;
    function<void(int)> dfs = [&dfs,&res](int v){
        for(int t=0; t<G[v].size(); t++){
            int i = G[v][t]; if(t != 0 && G[v][t-1] == G[v][t])
                continue;
            if(P[i] != v) continue; dfs(i);
            if((t+1 == G[v].size() || i != G[v][t+1]) && Low[i] >
                In[v]) res.emplace_back(min(v,i), max(v,i));
        }
    };
    for(int i=1; i<=n; i++) sort(G[i].begin(), G[i].end()); //
    multi edge -> sort
    for(int i=1; i<=n; i++) if(P[i] == -1) dfs(i);
    return move(res); // sort(all(res));
}
vector<int> BCC[MAX_V]; // BCC[v] = components which contains
v
void vertexDisjointBCC(int n){ // allow multi edge, not allow
self loop
    int cnt = 0; array<char,MAX_V> vis; vis.fill(0);
    function<void(int,int)> dfs = [&dfs,&vis,&cnt](int v, int
        c){
        vis[v] = 1; if(c > 0) BCC[v].push_back(c);
        for(auto i : G[v]){
            if(vis[i]) continue;
            if(In[v] <= Low[i]) BCC[v].push_back(++cnt), dfs(i,
                cnt);
            else dfs(i, c);
        }
    };
    for(int i=1; i<=n; i++) if(!vis[i]) dfs(i, 0);
    for(int i=1; i<=n; i++) if(BCC[i].empty())
        BCC[i].push_back(++cnt);
}

```

4.13 Find 3 or 4 cycle

Usage: Finds all C_3 and C_4 cycles using degree ordering.

Time Complexity: $O(M\sqrt{M})$

```

vector<tuple<int,int,int>> Find3Cycle(int n, const
vector<pair<int,int>> &edges){ // N+MsqrtN
    int m = edges.size();
    vector<int> deg(n), pos(n), ord; ord.reserve(n);
    vector<vector<int>> gph(n), que(m+1), vec(n);
    vector<vector<tuple<int,int,int>>> tri(n);
    vector<tuple<int,int,int>> res;

```

```

for(auto [u,v] : edges) deg[u]++, deg[v]++;
for(int i=0; i<n; i++) que[deg[i]].push_back(i);
for(int i=m; i>=0; i--) ord.insert(ord.end(),
    que[i].begin(), que[i].end());
for(int i=0; i<n; i++) pos[ord[i]] = i;
for(auto [u,v] : edges) gph[pos[u]].push_back(pos[v]),
    gph[pos[v]].push_back(pos[u]);
for(int i=0; i<n; i++){
    for(auto j : gph[i]){
        if(i > j) continue;
        for(int x=0, y=0; x<vec[i].size() && y<vec[j].size(); ){
            if(vec[i][x] == vec[j][y]) res.emplace_back(ord[i],
                ord[j], ord[vec[i][x]], x++, y++);
            else if(vec[i][x] < vec[j][y]) x++; else y++;
        }
        vec[j].push_back(i);
    }
}
for(auto &[u,v,w] : res){
    if(pos[u] < pos[v]) swap(u, v);
    if(pos[u] < pos[w]) swap(u, w);
    if(pos[v] < pos[w]) swap(v, w);
    tri[u].emplace_back(u, v, w);
}
res.clear();
for(int i=n-1; i>=0; i--) res.insert(res.end(),
    tri[ord[i]].begin(), tri[ord[i]].end());
return res;
}
bitset<500> B[500]; // N3/W
long long Count3Cycle(int n, const vector<pair<int,int>>
    &edges){
    long long res = 0;
    for(int i=0; i<n; i++) B[i].reset();
    for(auto [u,v] : edges) B[u].set(v), B[v].set(u);
    for(int i=0; i<n; i++) for(int j=i+1; j<n; j++)
        if(B[i].test(j)) res += (B[i] & B[j]).count();
    return res / 3;
}
// O(n + m * sqrt(m) + th) for graphs without loops or
multiedges
void Find4Cycle(int n, const vector<array<int, 2>> &edge,
    auto process, int th = 1){
    int m = (int)edge.size();
    vector<int> deg(n), order, pos(n);
    vector<vector<int>> appear(m+1), adj(n), found(n);
    for(auto [u, v]: edge) ++deg[u], ++deg[v];
    for(auto u=0; u<n; u++) appear[deg[u]].push_back(u);
    for(auto d=m; d>=0; d--) order.insert(order.end(),
        appear[d].begin(), appear[d].end());
    for(auto i=0; i<n; i++) pos[order[i]] = i;
    for(auto i=0; i<m; i++){
        int u = pos[edge[i][0]], v = pos[edge[i][1]];
        adj[u].push_back(v), adj[v].push_back(u);
    }
    T res = 0; vector<int> cnt(n);
    for(auto u=0; u<n; u++){

```

```

    for(auto v: adj[u]) if(u < v) for(auto w: adj[v]) if(u < w) cnt[w] = 0;
    for(auto v: adj[u]) if(u < v) for(auto w: adj[v]) if(u < w) res += cnt[w] ++;
}
for(auto u=0; u<n; u++){
    for(auto v: adj[u]) if(u < v) for(auto w: adj[v]) if(u < w) found[w].clear();
    for(auto v: adj[u]) if(u < v) for(auto w: adj[v]) if(u < w) {
        for(auto x: found[w]){
            if(!th--) return;
            process(order[u], order[v], order[w], order[x]);
        }
        found[w].push_back(v);
    }
}
}
}

```

4.14 Push Relabel

Usage: Max flow via preflow-push and labeling

Time Complexity: $O(V^3)$

```

template<typename T = ll> struct HLPP {
    struct Edge { int to; T cap; int rev; };
    T INF = numeric_limits<T>::max();
    vector<vector<Edge>> adj;
    vector<T> ex;
    vector<int> pos, lvl, nxt, gprv, gnxt;
    int n, high = 0, gap = 0, cnt = 0;
    HLPP(int n) : n(n+1), adj(n+2), ex(n+2), pos(n+2),
        lvl(n+2), nxt(2*n+3), gprv(2*n+3), gnxt(2*n+3) {}
    void add(int u, int v, T cap) {
        adj[u].push_back({ v, cap, sz(adj[v]) });
        adj[v].push_back({ u, 0, sz(adj[u])-1 });
    }
    void push(int u, int h) {
        nxt[u] = nxt[n+h]; nxt[n+h] = u;
        high = max(high, h);
    }
    void ugap(int u, int h) {
        gnxt[u] = gnxt[gprv[u] = n+h];
        gprv[gnxt[u]] = gnxt[gprv[u]] = u;
        gap = max(gap, h);
    }
    void dgap(int u) {
        gnxt[gprv[u]] = gnxt[u];
        gprv[gnxt[u]] = gprv[u];
    }
    void addex(int u, T f) {
        ex[u] += f;
        if (ex[u] == f) push(u, lvl[u]);
    }
    void upd(int u, int h) {
        if (lvl[u] != n+1) dgap(u);
        lvl[u] = h; if (h == n+1) return;
        ugap(u, h); if (ex[u] > 0) push(u, h);
    }
    void relabel(int t) {

```

```

        cnt = high = gap = 0;
        iota(nxt.begin()+n, nxt.end(), n);
        iota(gprv.begin()+n, gprv.end(), n);
        iota(gnxt.begin()+n, gnxt.end(), n);
        fill(all(lvl), n+1); fill(all(pos), 0);
        lvl[t] = 0; queue<int> q; q.push(t);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (auto &e : adj[u]) {
                if (!adj[e.to][e.rev].cap || lvl[e.to] <= lvl[u]+1)
                    continue;
                upd(e.to, lvl[u]+1); q.push(e.to);
            }
        }
    }
    void discharge(int u) {
        T &v = ex[u]; int h = n, Sz = sz(adj[u]);
        for (int &i = pos[u], k = Sz; k--; i = (i ? i : Sz)-1) {
            auto& e = adj[u][i];
            if (!e.cap) continue;
            if (lvl[u] != lvl[e.to]+1) {
                h = min(h, lvl[e.to]); continue;
            }
            auto f = min(v, e.cap);
            v -= f; addex(e.to, f); e.cap -= f;
            adj[e.to][e.rev].cap += f;
            if (!v) return;
        }
        cnt++;
        if (gnxt[gnxt[n+lvl[u]]] < n) {
            upd(u, h+1); return;
        }
        for (int i = lvl[u]; gap >= i; gap--) {
            while (gnxt[n+gap] < n) {
                int t = gnxt[n+gap];
                lvl[t] = n+1; dgap(t);
            }
        }
    }
    T flow(int s, int t) {
        relabel(t);
        addex(s, INF); ex[t] -= INF;
        while (~high) {
            int u = nxt[n+high];
            if (u >= n) { high--; continue; }
            nxt[n+high] = nxt[u];
            if (lvl[u] != high) continue;
            discharge(u);
            if (cnt >= 4*n) relabel(t);
        }
        return ex[t]+INF;
    }
};

```

4.15 Gomory Hu Tree

// Gomory-Hu Tree: 모든 정점 쌍의 최대 유량(Min-cut)을 표현하는 트리
 // 트리에서 u-v 경로 상의 최소 가중치 간선 = 원본 그래프에서 u-v의 최대 유량

```

struct Dinic {};
struct Edge { int u, v; ll w; };
vector<Edge> gomory_hu(int n, const vector<Edge> &ev) {
    vector<Edge> tree;
    vector<int> par(n+1, 1);
    for (int i = 2; i <= n; i++) {
        Dinic dn(n);
        for (auto& [u, v, w] : ev) { dn.add(u, v, w); dn.add(v, u, w); }
        tree.push_back({i, par[i], dn.flow(i, par[i])});
        vector<bool> cut = dn.mincut(i);
        for (int j = i + 1; j <= n; j++) {
            if (cut[j] && par[j] == par[i]) par[j] = i;
        }
    }
    return tree;
}
// ex. 인접정점간 최대유량 합이 최대인 순열
vector<int> par(n+1); iota(all(par), 0);
vector<vector<int>> p(n+1);
for (int i = 1; i <= n; i++) p[i] = {i};
function<int(int)> find = [&](int x) { return x == par[x] ? x : par[x] = find(par[x]); };
auto res = gomory_hu(n, edges);
sort(all(res), [](Edge& a, Edge& b) { return a.w > b.w; });
ll ans = 0;
for (auto &e : res) {
    int u = find(e.u), v = find(e.v);
    if (u == v) continue;
    ans += e.w; par[u] = v;
    p[v].insert(p[v].end(), all(p[u]));
}
// ans: 최대 유량의 합
// p[find(1)]: 조건을 만족하는 정점들의 순열 (크기 n)

```

5 DP Optimization

5.1 Convex Hull Trick

Usage: $dp[i] = \min(dp[j] + b[j] * a[i]), b[j] \geq b[j+1]$

Time Complexity: $O(N \log N)$

```

// Slopes(k) and queries(x) can be in any order
template <typename T> struct Line {
    mutable T k, m, p;
    bool operator<(const Line& o) const { return k < o.k; }
    bool operator<(T x) const { return p < x; }
};
template <typename T = ll> struct LineContainer :
    multiset<Line<T>, less<>> {
    using iterator = typename multiset<Line<T>,
        less<>>::iterator;
    // (for doubles, use div(a,b) = a/b)
    static constexpr T inf = numeric_limits<T>::max();
    T div(T a, T b) { // floored division
        return a / b - ((a ^ b) < 0 && a % b); }
    bool isect(iterator x, iterator y) {
        if (y == this->end()) return x->p = inf, 0;
        if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
    }
};

```

```

    else x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
}
void add(T k, T m) { // y = kx + m
    auto z = this->insert({k, m, 0}); y = z++, x = y;
    while (isect(y, z)) z = this->erase(z);
    if (x != this->begin() && isect(-x, y)) isect(x, y = this->erase(y));
    while ((y = x) != this->begin() && (--x)->p >= y->p) isect(x, this->erase(y));
}
T query(T x) {
    assert(!this->empty());
    auto l = *this->lower_bound(x);
    return l.k * x + l.m;
}
}; // add(-k, -m), -query(x) for Lower hull(min)
LineContainer CHT;
int main() {
    dp[0] = 0; CHT.add(a[0], dp[0]);
    for (int i = 1; i < n; i++) { // dp[i] = Max(a[j]*b[i] + dp[j])
        dp[i] = CHT.query(b[i]);
        CHT.add(a[i], dp[i]);
    }
    cout << dp[n-1] << "\n";
}

```

5.2 Linear CHT

Usage: CHT when slopes(k)/queries(x) are monotonic.

Time Complexity: $O(N)$

// slopes(k) and queries(x) must be monotonic

```

struct PLL {
    ll x, y;
    PLL(const ll x = 0, const ll y = 0) : x(x), y(y) {}
    bool operator<= (const PLL& i) const { return 1. * x / y
        <= 1. * i.x / i.y; }
};
struct ConvexHull {
    static ll F(const PLL& i, const ll x) { return i.x * x + i.y; }
    static PLL C(const PLL& a, const PLL& b) { return { a.y - b.y, b.x - a.x }; }
    deque<PLL> S;
    void add(const ll a, const ll b) {
        while (S.size() > 1 && C(S.back(), PLL(a, b)) <= C(S[S.size() - 2], S.back())) S.pop_back();
        S.push_back(PLL(a, b));
        /* when x is monotonic decreasing
        while (S.size() > 1 && C(S[0], S[1]) <= C(PLL(a, b), S[0])) S.pop_front();
        S.push_front(PLL(a, b)); */
    }
    ll query(const ll x) {
        while (S.size() > 1 && F(S[0], x) <= F(S[1], x)) S.pop_front(); // upper hull(max)
        // while (S.size() > 1 && F(S[0], x) >= F(S[1], x)) S.pop_front(); // lower hull(min)
    }
}

```

```

    return F(S[0], x);
}
} CHT;

```

5.3 D&C optimization

Usage: dp[t][i] = min(dp[t-1][j] + c[j][i]), c is Monge

Time Complexity: $O(KN \log N)$

```

ll dp[MAX_K][MAX_N];
// 1부터 r까지 구간의 비용을 계산하는 함수
ll get_cost(int l, int r) { /* return sum[l][r] + C; */
// k: 현재단계(구간개수 등), pL, pR: j를 찾을 탐색범위
void dnc(int k, int l, int r, int pL, int pR) {
    if (l > r) return;
    int opt = pL, mid = (l + r) / 2;
    dp[k][mid] = -1e18 // 최솟값문제: INF, 최댓값: -INF
    for (int j = pL; j <= min(mid, pR); j++) {
        ll val = (j==0 ? dp[k-1][j-1]) + get_cost(j, mid);
        if (val > dp[k][mid]) {
            dp[k][mid] = val; opt = j;
        }
    }
    dnc(k, l, mid - 1, pL, opt);
    dnc(k, mid + 1, r, opt, pR);
}
// for (int i = 1; i <= T; i++) dnc(i, 0, n-1, 0, n-1);
}

```

5.4 Monotone Queue optimization

Usage: dp[i] = min(dp[j] + c[j][i]), c is Monge, find cross

Time Complexity: $O(N \log N)$

```

ll f(int j, int i); // j->i로 전이 값 (dp[j] + cost(j, i))
void solve() {
    auto cross = [&](ll p, ll q) {
        ll lo = max(p, q), hi = n + 1;
        while (lo + 1 < hi) {
            ll mid = (lo + hi) / 2;
            if (f(p, mid) > f(q, mid)) hi = mid; // min 기준: f(p) > f(q)면 q가 우세
            else lo = mid;
        }
        return hi;
    };
    deque<pair<ll, ll>> dq; // {cand_idx, start_pos}
    dq.push_back({0, 1}); // 초기값: 0이 1번 위치부터 최적가정
    for (int i = 1; i <= n; i++) {
        while (dq.size() > 1 && dq[1].second <= i) dq.pop_front();
        dp[i] = f(dq[0].first, i);
        while (!dq.empty()) {
            ll p = dq.back().first, pos = cross(p, i);
            if (pos <= dq.back().second) dq.pop_back();
            else {
                if (pos <= n) dq.push_back({i, pos});
                break;
            }
        }
        if (dq.empty()) dq.push_back({i, 1});
    }
}
}

```

5.5 Aliens Trick

Usage: dp[t][i] = min(dp[t-1][j] + c[j+1][i]), c is Monge, find lambda w/ half bs

Time Complexity: $O(T \log X)$

```

/* n: 원소 개수 (경로 복원 끝점), k: 골라야 하는 개수
* lo, hi: 패널티 이분탐색 범위 (0 ~ 최대 가치)
* f(c): 패널티가 c일 때 {2*(가치합), prv} 반환 (가치는 c를 뺀 값) */
template<class T, bool GET_MAX = false>
pair<T, vector<int>> AliensTrick(int n, int k, auto f, T lo, T hi) {
    T l = lo, r = hi;
    while (l < r) {
        T m = (l + r + (GET_MAX ? 1 : 0)) >> 1;
        vector<int> prv = f(m * 2 + (GET_MAX ? -1 : 1)).second;
        int cnt = 0; for (int i = n; i; i = prv[i]) cnt++;
        if (cnt <= k) (GET_MAX ? l : r) = m;
        else (GET_MAX ? r : l) = m + (GET_MAX ? -1 : 1);
    }
    T opt_val = f(l * 2).first / 2 - k * l;
    auto get_path = [&](T c) {
        vector<int> p{n};
        for (auto prv = f(c).second; p.back(); )
            p.push_back(prv[p.back()]);
        reverse(p.begin(), p.end()); return p;
    };
    auto p1 = get_path(l * 2 + (GET_MAX ? 1 : -1));
    auto p2 = get_path(l * 2 - (GET_MAX ? 1 : -1));
    if (p1.size() == k + 1) return {opt_val, p1};
    if (p2.size() == k + 1) return {opt_val, p2};
    for (int i = 1, j = 1; i < p1.size(); i++) {
        while (j < p2.size() && p2[j] < p1[i - 1]) j++;
        if (p1[i] <= p2[j] && i - j == k + 1 - (int)p2.size()) {
            vector<int> res(p1.begin(), p1.begin() + i);
            res.insert(res.end(), p2.begin() + j, p2.end());
            return {opt_val, res};
        }
    }
    return {opt_val, {}}; // Should not reach here
}

```

5.6 Sum Over Subsets

Usage: dp[mask] = sum(A[i]), i is in mask

Time Complexity: $O(N2^N)$

```

// 1. Subset Sum: f[mask] = sum of a[sub] where sub & mask == sub
for (int i = 0; i < n; i++)
    for (int mask = 0; mask < (1 << n); mask++)
        if (mask & (1 << i)) f[mask] += f[mask ^ (1 << i)];
// 2. Superset Sum: f[mask] = sum of a[sup] where sup & mask == mask
for (int i = 0; i < n; i++)
    for (int mask = (1 << n) - 1; mask >= 0; mask--)
        if (!(mask & (1 << i))) f[mask] += f[mask ^ (1 << i)];

```

5.7 Berlekamp Massey

Usage: Linear recurrence N -th term. Sparse matrix determinant.

Time Complexity: N -th term: $O(K^2 \log N)$ / Det: $O(N(N + E))$

```
mt19937_64
rng(chrono::steady_clock::now().time_since_epoch().count());
int randint(int lb, int ub) { return
uniform_int_distribution<int>(lb, ub)(rng); }
const int mod = 998244353;
ll ipow(ll x, ll p) {
    ll ret = 1, piv = x;
    while (p) {
        if (p & 1) ret = ret * piv % mod;
        piv = piv * piv % mod; p >>= 1;
    }
    return ret;
}
vector<int> berlekamp_massey(vector<int> x) {
    vector<int> ls, cur; int lf, ld;
    for (int i = 0; i < sz(x); i++) {
        ll t = 0;
        for (int j = 0; j < sz(cur); j++) t = (t +
11l*x[i-j-1]*cur[j]) % mod;
        if ((t - x[i]) % mod == 0) continue;
        if (cur.empty()) {
            lf = i; ld = (t-x[i]) % mod;
            cur.resize(i+1); continue;
        }
        ll k = -(x[i]-t) * ipow(ld, mod-2) % mod;
        vector<int> c(i-lf-1); c.push_back(k);
        for (auto& j : ls) c.push_back((-j*k % mod));
        if (sz(c) < sz(cur)) c.resize(sz(cur));
        for (int j = 0; j < sz(cur); j++) c[j] = (c[j]+cur[j]) % mod;
        if (i-lf+sz(ls) >= sz(cur)) {
            tie(ls, lf, ld) = make_tuple(cur, i, (t-x[i]) % mod);
            cur = c;
        }
        for (auto& i : cur) i = (i % mod + mod) % mod;
        return cur;
    }
}
int get_nth(vector<int> rec, vector<int> dp, ll n) {
    int m = sz(rec); vector<int> s(m), t(m); s[0] = 1;
    if (m != 1) t[1] = 1; else t[0] = rec[0];
    auto mul = [&rec](vector<int> v, vector<int> w) {
        int m = sz(v); vector<int> t(2*m);
        for (int j = 0; j < m; j++) {
            for (int k = 0; k < m; k++) {
                t[j+k] += 11l*v[j]*w[k] % mod;
                if (t[j+k] >= mod) t[j+k] -= mod;
            }
        }
        for (int j = 2*m-1; j >= m; j--) {
            for (int k = 1; k <= m; k++) {
                t[j-k] += 11l*t[j]*rec[k-1] % mod;
                if (t[j-k] >= mod) t[j-k] -= mod;
            }
        }
        t.resize(m); return t;
    };
};
```

```
while (n) {
    if (n & 1) s = mul(s, t);
    t = mul(t, t); n >>= 1;
}
ll ret = 0;
for (int i = 0; i < m; i++) ret += 11l*s[i]*dp[i] % mod;
return ret % mod;
}
int guess_nth_term(vector<int> x, ll n) {
    if (n < sz(x)) return x[n];
    vector<int> v = berlekamp_massey(x);
    if (v.empty()) return 0;
    return get_nth(v, x, n);
}
struct elem { int x, y, v; }; // A_(x, y) <- v, 0-based. no
duplicate
vector<int> get_min_poly(int n, vector<elem> M) {
    // smallest poly P such that A^i = sum_{j < i} {A^j \times
P_j}
    vector<int> rnd1, rnd2;
    for (int i = 0; i < n; i++) {
        rnd1.push_back(randint(1, mod - 1));
        rnd2.push_back(randint(1, mod - 1));
    }
    vector<int> gobs;
    for (int i = 0; i < 2*n+2; i++) {
        int tmp = 0;
        for (int j = 0; j < n; j++) {
            tmp += 11l * rnd2[j] * rnd1[j] % mod;
            if (tmp >= mod) tmp -= mod;
        }
        gobs.push_back(tmp); vector<int> nxt(n);
        for (auto& i : M) {
            nxt[i.x] += 11l * i.v * rnd1[i.y] % mod;
            if (nxt[i.x] >= mod) nxt[i.x] -= mod;
        }
        rnd1 = nxt;
    }
    auto sol = berlekamp_massey(gobs);
    reverse(all(sol)); return sol;
}
ll det(int n, vector<elem> M) {
    vector<int> rnd;
    for (int i = 0; i < n; i++) rnd.push_back(randint(1,
mod-1));
    for (auto& i : M) i.v = 11l * i.v * rnd[i.y] % mod;
    auto sol = get_min_poly(n, M)[0]; if (n%2 == 0) sol =
mod-sol;
    for (auto& i : rnd) sol = 11l * sol * ipow(i, mod-2) % mod;
    return sol;
}
}
```

6 Geometry

6.1 Geometry Template

Usage: Basic point, line, and circle operations.

```
const double EPS = 1e-9;
template<typename T> struct Point {
    T x, y;
```

```
bool operator<(const Point& p) const { return x==p.x ?
y<p.y : x<p.x; }
bool operator==(const Point& p) const { return x==p.x &&
y==p.y; }
Point operator+(const Point& p) const { return {x+p.x,
y+p.y}; }
Point operator-(const Point& p) const { return {x-p.x,
y-p.y}; }
Point operator*(T n) const { return {x*n, y*n}; }
Point operator/(T n) const { return {x/n, y/n}; }
T operator*(const Point& p) const { return x*p.x + y*p.y; }
T operator/(const Point& p) const { return x*p.y - y*p.x; }
/* 3D dot, cross
T operator*(const Point& p) const { return x*p.x + y*p.y +
z*p.z; }
Point operator/(const Point& p) const {
    return {y*p.z - z*p.y, z*p.x - x*p.z, x*p.y - y*p.x};
} */
T dist2() const { return x*x + y*y; }
Point<double> d() const { return {(double)x, (double)y}; }
Point rot90() const { return {-y, x}; }
};
using P = Point<ll>;
using Pd = Point<double>;
int ccw(P a, P b, P c) {
    ll res = (b - a) / (c - a);
    return (res > 0) - (res < 0);
}
bool isInter(P a, P b, P c, P d) {
    int ab = ccw(a, b, c) * ccw(a, b, d), cd = ccw(c, d, a) *
ccw(c, d, b);
    if (ab == 0 && cd == 0) {
        if (b < a) swap(a, b); if (d < c) swap(c, d);
        return !(b < c || d < a);
    }
    return ab <= 0 && cd <= 0;
}
ll area2(const vector<P>& v) { // 다각형 넓이*2
    ll res = 0; int n = sz(v);
    for (int i = 0; i < n; i++) res += v[i] / v[(i+1)%n];
    return abs(res);
}
}
```

6.2 Closest Pair

Time Complexity: $O(N \log N)$

```
struct P { ll x, y; int id; };
ll dist(const P &a, const P &b) {
    return (a.x-b.x)*(a.x-b.x) + (a.y-b.y)*(a.y-b.y);
}
array<ll,3> closestPair(vector<P> &v) {
    sort(all(v), [](const P &a, const P &b) { return a.x <
b.x; });
    auto cmp = [](const P &a, const P &b) {
        return a.y == b.y ? a.x < b.x : a.y < b.y;
    };
    set<P, decltype(cmp)> s(cmp);
```



```

ll d = 9e18; array<ll,3> ans = { d, -1, -1 };
for (int i = 0, j = 0; i < sz(v); i++) {
    while (j < i && (v[i].x-v[j].x)*(v[i].x-v[j].x) >= d)
        s.erase(v[j++]);
    ll dsq = sqrt(d)+1;
    auto it1 = s.lower_bound({ LLONG_MIN, v[i].y - dsq, -1 });
    auto it2 = s.upper_bound({ LLONG_MAX, v[i].y + dsq, -1 });
    for (auto it = it1; it != it2; it++) {
        ll cur = dist(v[i], *it);
        if (cur < d) ans = { d = cur, v[i].id, (*it).id };
    }
    s.insert(v[i]);
}
return ans;
}

```

6.3 Convex Hull

Usage: Finds the convex hull using Monotone Chain. Returns vertices in CCW order.

Time Complexity: $O(N \log N)$

```

vector<P> ConvexHull(vector<P> ps) { // Monotone chain
    compress(ps); if (sz(ps) <= 2) return ps;
    vector<P> v(sz(ps)*2);
    int s = 0, t = 0;
    for (int i = 2; i--; s = --t, reverse(all(ps))) {
        for (P p : ps) { // include boundary : ccw < 0
            while (t >= s+2 && ccw(v[t-2], v[t-1], p) <= 0) t--;
            v[t++] = p;
        }
    }
    v.resize(t + (t <= 1)); return v;
}

```

6.4 Convex Hull 3d

Time Complexity: $O(N^2)$

```

struct face{ int a, b, c; PT d; };
vector<face> convex_hull_3d(vector<PT> &p) {
    // -1. 중복 점 제거 (normalize하면 중복 점이 생길 수 있음)
    // 0. size <= 3 -> exit
    // 1. 전부 한 직선 위에 있는지 판정 yes -> exit
    // 2. first 3 points are not on the same line일 때까지
    //    스펙하고 전부 한 평면 위에 있는지 판정 yes -> exit
    int n = p.size();
    if (n <= 3) exit(1);
    while (true) {
        // shuffle until first 4 points are not on the same plane
        shuffle(all(p), mt19937(random_device{}()));
        if (((p[1]-p[0])^(p[2]-p[0])) * (p[3]-p[0]) != 0) break;
    }
    vector<face> f;
    vector<vector<bool>> dead(n, vector<bool>(n, true));
    auto add_face = [&](int a, int b, int c) {
        f.push_back({a, b, c, (p[b]-p[a]) ^ (p[c]-p[a])});
        dead[a][b] = dead[b][c] = dead[c][a] = 0;
    };
    add_face(0, 1, 2); add_face(0, 2, 1);
    for (int i = 3; i < n; i++) {
        vector<face> nf;

```

```

        for (face &F : f){
            if ((p[i] - p[F.a]) * F.d > 0) {
                dead[F.a][F.b] = dead[F.b][F.c] = dead[F.c][F.a] = 1;
            } else {
                nf.push_back(F);
            }
        }
        f = nf;
        for (face &F : nf){
            if (dead[F.b][F.a]) add_face(F.b, F.a, i);
            if (dead[F.c][F.b]) add_face(F.c, F.b, i);
            if (dead[F.a][F.c]) add_face(F.a, F.c, i);
        }
    }
    return f;
}

```

6.5 Rotating Calipers

Usage: Calculates the diameter (farthest pair of points) of a convex hull (CCW order).

Time Complexity: $O(N)$

```

// ccw 정렬된 convex에서 가장 먼 두 점
pair<P,P> Calipers(const vector<P>& v) {
    int n = sz(v); if (n < 2) return {v[0], v[0]};
    ll mx = 0; P a = v[0], b = v[1];
    for (int i = 0, j = 1; i < n; i++) {
        P t = v[(i+1)%n] - v[i];
        while ((t / (v[(j+1)%n] - v[j])) > 0) j = (j+1)%n;
        ll now = (v[i] - v[j]).dist2();
        if (now > mx) mx = now, a = v[i], b = v[j];
    }
    return {a, b};
}

```

6.6 Point in Convex Polygon

Usage: Checks if a point is inside or on the boundary of a convex polygon (CCW sorted).

Time Complexity: $O(\log N)$

```

// CCW 정렬된 다각형 내부 판별,  $O(\log N)$ 
bool InConvPoly(const vector<P>& v, P p) {
    int n = sz(v); if (n<3) return false;
    // ccw <= 0 || ccw >= 0: exclude boundary
    if (ccw(v[0], v[1], p) < 0 || ccw(v[0], v.back(), p) > 0)
        return false;
    int l = 1, r = n - 1;
    while (l + 1 < r) {
        int mid = (l + r) / 2;
        if (ccw(v[0], v[mid], p) >= 0) l = mid;
        else r = mid;
    }
    return ccw(v[1], v[l + 1], p) >= 0; // > 0: exclude boundary
}

```

6.7 Point in Polygon

Usage: Ray casting algorithm for general polygons.

Time Complexity: $O(N)$

// 다각형 내부 판별 (CCW/CW 순서 무관)

```

bool InPoly(const vector<P>& v, P p) {
    int n = sz(v); bool chk = false;
    for (int i = 0; i < n; i++) {
        P a = v[i], b = v[(i+1) % n];
        if ((b-a) / (p-a) == 0 && (a-p) * (b-p) <= 0) return
            true; // false: exclude boundary
        if ((a.y > p.y) != (b.y > p.y)) {
            double ix = (double)(b.x-a.x) * (p.y-a.y) /
                (double)(b.y-a.y) + a.x;
            if (p.x < ix) chk = !chk;
        }
    }
    return chk;
}

```

6.8 Sort Points

Usage: Angular sort. 1. Relative to pivot (Convex Hull). 2. Relative to origin.

```

// Sort by angle relative to the bottom-left point (CCW)
void Sort(vector<P> &v) {
    if (v.size() < 2) return;
    swap(v[0], *min_element(all(v), [](const P& a, const P& b) {
        return a.y != b.y ? a.y < b.y : a.x < b.x;
    }));
    sort(v.begin() + 1, v.end(), [&](const P& a, const P& b) {
        ll cp = (a-v[0]) / (b-v[0]); if (cp != 0) return cp > 0;
        return (a - v[0]).dist2() < (b - v[0]).dist2();
    });
}
// Sort by angle around a given point in the range [0, 360)
void Sort(vector<P> &v, P cent = { 0, 0 }) {
    auto half = [](const P& p) { return p.y > 0 || (p.y == 0
        && p.x > 0); };
    sort(all(v), [&](const P& a, const P& b) {
        P aa = a-cent, bb = b-cent;
        if (half(aa) != half(bb)) return half(aa) > half(bb);
        return (aa / bb) > 0;
    });
}

```

6.9 Linear Minkowski Sum

Usage: Minkowski Sum of Two convex (must be CCW order).

Time Complexity: $O(N + M)$

```

vector<P> Minkowski(vector<P> p, vector<P> q) {
    if (p.empty() || q.empty()) return {};
    rotate(p.begin(), min_element(all(p)), p.end());
    rotate(q.begin(), min_element(all(q)), q.end());
    p.push_back(p[0]); p.push_back(p[1]);
    q.push_back(q[0]); q.push_back(q[1]);
    vector<P> res; int i = 0, j = 0;
    while (i < sz(p) - 2 || j < sz(q) - 2) {
        res.push_back(p[i] + q[j]);
        ll cp = (p[i + 1] - p[i]) / (q[j + 1] - q[j]);
        if (cp >= 0 && i < sz(p) - 2) i++;
        if (cp <= 0 && j < sz(q) - 2) j++;
    }
    return res;
}

```


6.10 Smallest Enclosing Circle

Usage: Welzl's algorithm to find the Minimum Enclosing Circle.

Returns center, radius.

Time Complexity: Expected $O(N)$

```
Pd getC(Pd a, Pd b) { return (a + b) / 2.0; }
Pd getC(Pd a, Pd b, Pd c) {
    Pd u = b-a, v = c-a; double d = 2*(u/v);
    if (abs(d) < EPS) return {0, 0};
    return a - (v * u.dist2() - u * v.dist2()).rot90() / d;
}
/* // for 3D
Pd getC(Pd a, Pd b, Pd c) {
    Pd u = b-a, v = c-a, n = u/v;
    if (n.dist2() < EPS) return getC(a, b);
    Pd top = (n/u) * v.dist2() + (v/n) * u.dist2();
    return a + top / (2*n.dist2());
}
Pd getC(Pd a, Pd b, Pd c, Pd d) {
    Pd p = b-a, q = c-a, r = d-a;
    double det = ((p / q) * r) * 2.0;
    if (abs(det) < EPS) return a;
    Pd top = (p/q) * r.dist2() + (q/r) * p.dist2() + (r/p) *
        q.dist2();
    return a + top / det;
}
*/
mt19937_64
rng(chrono::steady_clock::now().time_since_epoch().count());
pair<Pd, double> min_circle(vector<Pd> v) {
    shuffle(all(v), rng);
    Pd c = {0, 0}; double r = 0;
    auto dist = [&](Pd p1, Pd p2) { return
        sqrt((p1-p2).dist2()); };
    auto chk = [&](Pd p) { return dist(c, p) > r+EPS; };
    for (int i = 0; i < sz(v); i++) if (chk(v[i])) {
        c = v[i]; r = 0;
        for (int j = 0; j < i; j++) if (chk(v[j])) {
            c = getC(v[i], v[j]); r = dist(c, v[i]);
            for (int k = 0; k < j; k++) if (chk(v[k])) {
                c = getC(v[i], v[j], v[k]); r = dist(c, v[k]);
            }
        }
    }
    return {c, r};
}
```

6.11 Geometric Intersections

Usage: Intersection primitives: Segment, Line-Circle, Line-Hull, and Circle-Polygon area.

Time Complexity: $O(1)/O(1)/O(\log N)/O(N)$

```
using T = _int128_t; // T <= 0(COORD^3)
// [1] 선분 교차 (정밀 좌표) / Param: 선분 ab, 선분 cd
// Return: {flag, xn, xd, yn, yd} -> 교점 (xn/xd, yn/yd)
// Flag: 0(안만남), 1(교점=끝점), 4(교차), -1(무수히 겹침)
tuple<int, T, T, T, T> segmentInter(P a, P b, P c, P d) {
```

```
    if (!isInter(a, b, c, d)) return {0, 0, 0, 0, 0};
    T det = (b - a) / (d - c);
    if (det == 0) {
        if (b < a) swap(a, b);
        if (d < c) swap(c, d);
        if (b == c) return {1, b.x, 1, b.y, 1};
        if (d == a) return {1, d.x, 1, d.y, 1};
        return {-1, 0, 0, 0, 0};
    }
    T p = (c - a) / (d - c), q = det;
    T xp = a.x * q + (b.x - a.x) * p, xq = q;
    T yp = a.y * q + (b.y - a.y) * p, yq = q;
    if (xq < 0) { xp = -xp; xq = -xq; }
    if (yq < 0) { yp = -yp; yq = -yq; }
    T g_x = __gcd(xp, xq); xp /= g_x; xq /= g_x;
    T g_y = __gcd(yp, yq); yp /= g_y; yq /= g_y;
    int f = 4;
    if ((xp == a.x * xq && yp == a.y * yq) || (xp == b.x * xq
        && yp == b.y * yq)) f = 1;
    if ((xp == c.x * xq && yp == c.y * yq) || (xp == d.x * xq
        && yp == d.y * yq)) f = 1;
    return {f, xp, xq, yp, yq};
}
// [2] 원-직선 교차 / Param: 직선 ab, 원(c, r)
// Return: 교점 좌표 목록 (a -> b 방향 순서 정렬됨)
vector<Pd> lineCircle(P a, P b, P c, double r) {
    P ab = b - a;
    Pd p = a.d() + ab.d() * ((c - a) * ab / (double)ab.dist2());
    double h2 = r * r - (p - c.d()).dist2();
    if (h2 < -EPS) return {};
    if (abs(h2) < EPS) return {p};
    Pd h = ab.d() * (sqrt(h2) / sqrt(ab.dist2()));
    return {p - h, p + h};
}
// [3] 원-다각형 교차 넓이
// Param: 원(c, r), 다각형 poly (CCW/CW 무관) Return: 교차하는
// 영역의 넓이
double areaCirclePoly(P c, double r, const vector<P>& poly) {
    auto tri = [&](Pd p, Pd q) {
        Pd d = q - p;
        double a = d * p / d.dist2(), b = (p.dist2() - r * r) /
            d.dist2();
        double det = a * a - b;
        if (det <= EPS) return r * r * atan2(p / q, p * q);
        double t1 = -a-sqrt(max(0.0, det)), t2 =
            -a+sqrt(max(0.0, det));
        if (t2 < -EPS || t1 > 1.0 + EPS) return r * r * atan2(p
            / q, p * q);
        Pd u = p + d * max(0.0, t1), v = p + d * min(1.0, t2);
        return r * r * atan2(p / u, p * u) + u / v + r * r *
            atan2(v / q, v * q);
    };
    double res = 0; int n = poly.size();
    for(int i = 0; i < n; i++)
        res += tri((poly[i] - c).d(), (poly[(i+1) % n] - c).d());
    return res * 0.5;
}
// [4] 볼록 다각형-직선 교차 ( $O(\log N)$ ) Param: 볼록 다각형 h
// (CCW), 직선 ab
```

```
// Return: {i, j} -> 직선이 변(i, i+1)과 변(j, j+1)을 교차함.
// 안만나면 {-1, -1}
int extrVertex(const vector<P>& h, P dir) {
    int n = h.size();
    auto cmp = [&](int i, int j) { return ccw({0,0}, dir,
        h[i%n] - h[j%n]); };
    auto isExtr = [&](int i) { return cmp(i, i - 1 + n) >= 0
        && cmp(i, i + 1) > 0; };
    if (isExtr(0)) return 0;
    int l = 0, r = n;
    while (l + 1 < r) {
        int m = (l + r) / 2;
        if (isExtr(m)) return m;
        if (cmp(l + 1, l) > 0) {
            if (cmp(m + 1, m) > 0 && cmp(l, m) > 0) r = m; else l
                = m;
        } else {
            if (cmp(m + 1, m) <= 0 && cmp(l, m) < 0) l = m; else r
                = m;
        }
    }
    return l;
}
array<int, 2> hullLineInter(const vector<P>& h, P a, P b) {
    P dir = b - a, per = {-dir.y, dir.x};
    int n = h.size();
    int i1 = extrVertex(h, per), i2 = extrVertex(h, per * -1);
    if (ccw(a, b, h[i1]) * ccw(a, b, h[i2]) > 0) return {-1,
        -1};
    auto search = [&](int s, int e) {
        if (ccw(a, b, h[s]) == 0) return s;
        int l = s, r = e;
        if (r < l) r += n;
        while (l + 1 < r) {
            int m = (l + r) / 2;
            if (ccw(a, b, h[m % n]) == ccw(a, b, h[s])) l = m;
            else r = m;
        }
        return l % n;
    };
    return {search(i1, i2), search(i2, i1)};
}
```

6.12 Half Plane Intersection

Usage: Intersection of half-planes defined by lines (left side is valid). Returns a convex polygon.

Time Complexity: $O(N \log N)$

// 각 선분의 '왼쪽' 영역들의 교집합(볼록 다각형)을 반환
// 영역이 없거나 닫히지 않는 경우(Unbounded) 빈 벡터 반환
가능성 있음

```
struct Line {
    double a, b, c; // ax + by <= c
    Line(Pd p1, Pd p2) {
        a = p1.y - p2.y; // -dy
        b = p2.x - p1.x; // dx
        c = a * p1.x + b * p1.y;
    }
    Pd slope() const { return {a, b}; }
```

```

};
Pd intersect(Line u, Line v) { // 평행하지 않은 두 직선의 교점
    double det = u.a * v.b - u.b * v.a;
    return {(u.c * v.b - u.b * v.c) / det, (u.a * v.c - u.c *
        v.a) / det};
}
bool bad(Line l, Pd p) {
    return l.a * p.x + l.b * p.y > l.c + EPS;
}
vector<Pd> HPI(vector<Line> lines) {
    sort(all(lines), [&](const Line& u, const Line& v) {
        Pd p1 = u.slope(), p2 = v.slope();
        bool f1 = p1.y > 0 || (p1.y == 0 && p1.x > 0);
        bool f2 = p2.y > 0 || (p2.y == 0 && p2.x > 0);
        if (f1 != f2) return f1 > f2;
        if (abs(p1 / p2) > EPS) return (p1 / p2) > 0;
        return u.c < v.c;
    });
    deque<Line> dq;
    for (auto& l : lines) {
        if (!dq.empty() && abs(dq.back().slope() / l.slope()) <
            EPS) continue;
        while (sz(dq) >= 2 && bad(l, intersect(dq.back(),
            dq[sz(dq)-2]))) dq.pop_back();
        while (sz(dq) >= 2 && bad(l, intersect(dq[0], dq[1])))
            dq.pop_front();
        dq.push_back(l);
    }
    while (sz(dq) > 2 && bad(dq[0], intersect(dq.back(),
        dq[sz(dq)-2]))) dq.pop_back();
    while (sz(dq) > 2 && bad(dq.back(), intersect(dq[0],
        dq[1]))) dq.pop_front();
    vector<Pd> res; if (sz(dq) < 3) return {};
    for (int i = 0; i < sz(dq); i++) {
        res.push_back(intersect(dq[i], dq[(i + 1) % sz(dq)]));
    }
    return res;
}

```

6.13 Manhattan MST

Time Complexity: $O(N \log N)$

```

vector<array<ll, 3>> manhattan_mst(polygon v) {
    vector<int> idx(v.size());
    for (int i = 0; i < idx.size(); i++) idx[i] = i;
    vector<array<ll, 3>> edges;
    for (int rot = 0; rot < 4; rot++) {
        sort(idx.begin(), idx.end(), [&v](int i, int j){
            return (v[i].x + v[i].y) < (v[j].x + v[j].y);
        });
        map<int, int, greater<int>> active;
        for (auto i : idx) {
            for (auto it = active.lower_bound(v[i].x); it !=
                active.end(); active.erase(it++)) {
                int j = it->second;
                if (v[i].x - v[i].y > v[j].x - v[j].y) break;
                edges.push_back({v[i].x - v[j].x + (v[i].y -
                    v[j].y), i+1, j+1});
            }
        }
    }
}

```

```

        active[v[i].x] = i;
    }
    for (auto &p : v) {
        if (rot & 1) p.x *= -1;
        else swap(p.x, p.y);
    }
    sort(all(edges));
    return edges;
}

```

7 String

7.1 Aho-Corasick

Usage: Multi-pattern matching using trie and failure links.

Time Complexity: $O(\sum |P| + |T|)$

// Aho-Corasick: 여러 패턴을 T에서 한 번에 매칭
 // ac.insert(p, id) -> ac.build() -> ac.find(T)/ac.count(T)
 // find: {패턴id, 끝idx} 목록(겹치는 짧은 패턴은 out 링크로).
 id는 1부터.

```

const int CH = 26;
struct AhoCorasick {
    struct Node {
        int ch[CH], nxt[CH], id, fail, out, cnt;
        Node() { memset(ch, 0, sizeof ch);
            memset(nxt, 0, sizeof nxt);
            id = fail = out = cnt = 0;
        }
    };
    vector<Node> T; int tc = 1;
    AhoCorasick(int SZ) { T.resize(SZ); }
    int ID(char c) { return c - 'a'; }
    void insert(const string &s, int id) {
        int x = 1; // 'id' must be >= 1
        for (auto c : s) {
            int cid = ID(c);
            if (!T[x].ch[cid]) T[x].ch[cid] = ++tc;
            x = T[x].ch[cid];
        }
        T[x].id = id; T[x].cnt++;
    }
    int next(int x, int c) {
        int& res = T[x].nxt[c]; if (res) return res;
        if (x != 1 && !T[x].ch[c]) res = next(T[x].fail, c);
        else if (T[x].ch[c]) res = T[x].ch[c];
        else res = 1;
        return res;
    }
    void build() {
        queue<int> q;
        for (int i = 0; i < CH; i++) {
            if (int c = T[1].ch[i]) T[c].fail = 1, q.push(c);
        }
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (int i = 0; i < CH; i++) {
                int c = T[v].ch[i]; if (!c) continue;
                int fail = T[c].fail = next(T[v].fail, i);
                if (T[fail].id) T[c].out = fail;
            }
        }
    }
}

```

```

        else T[c].out = T[fail].out;
        T[c].cnt += T[fail].cnt; q.push(c);
    }
}
vector<pair<int, int>> find(const string &s) {
    vector<pair<int, int>> res;
    int x = 1, tot = 0;
    for (int i = 0; i < sz(s); i++) {
        x = next(x, ID(s[i])); tot += T[x].cnt;
        if (T[x].id) res.emplace_back(T[x].id, i);
        for (int y = T[x].out; y; y = T[y].out)
            res.emplace_back(T[y].id, i);
    }
    assert(sz(res) == tot);
    return res;
}
ll count(const string &s) {
    ll res = 0; int x = 1;
    for (char c : s) res += T[x = next(x, ID(c))].cnt;
    return res;
}
};

```

7.2 Hashing

Usage: Rolling hash for string matching.

Time Complexity: $O(N)$

```

template<ll P1, ll M1, ll P2, ll M2>
struct Hashing {
    vector<ll> h1, h2, p1, p2;
    void build(const string &s) {
        int n = sz(s);
        h1 = h2 = p1 = p2 = vector<ll>(n+1);
        p1[0] = p2[0] = 1;
        for (int i = 0; i < n; i++) {
            h1[i+1] = (h1[i] * P1 + s[i]) % M1;
            h2[i+1] = (h2[i] * P2 + s[i]) % M2;
            p1[i+1] = p1[i] * P1 % M1;
            p2[i+1] = p2[i] * P2 % M2;
        }
    }
    pair<ll, ll> get(int l, int r) const { // 0-idx, [l, r)
        ll r1 = (h1[r] - h1[l] * p1[r-l]) % M1;
        ll r2 = (h2[r] - h2[l] * p2[r-l]) % M2;
        if (r1 < 0) r1 += M1;
        if (r2 < 0) r2 += M2;
        return { r1, r2 };
    }
};
// 1e5+3, 1e5+13, 131'071, 524'287, 1'299'709, 1'301'021
// 1e9-63, 1e9+7, 1e9+9, 1e9+103
// using Hash = Hashing<917, 998244353, 10009, 1000000007>;
// HS H; H.build(s);
// H.get(l, r) == H.get(l2, r2): s[l..r-1] == s[l2..r2-1]

```

7.3 KMP

Usage: Single pattern matching using prefix function.

Time Complexity: $O(N + M)$

```
// minPeriod(): 최소 주기. s = p^k 꼴이면 |p|, 아니면 len(s)
// pi[i]: prefix [0..i]의 가장 접두사=접미사 길이
template<typename T> struct KMP {
    T P; vector<int> pi;
    KMP(const T& P) : P(P), pi(sz(P)) {
        for (int i = 1, j = 0; i < sz(P); i++) {
            while (j > 0 && P[i] != P[j]) j = pi[j-1];
            if (P[i] == P[j]) pi[i] = ++j;
        }
    }
    vector<int> find(const T& S) {
        vector<int> res;
        int n = sz(S), m = sz(P), j = 0;
        for (int i = 0; i < n; i++) {
            while (j > 0 && S[i] != P[j]) j = pi[j-1];
            if (S[i] == P[j]) {
                if (j == m-1)
                    res.push_back(i-m+1), j = pi[j];
                else j++;
            }
        }
        return res;
    }
    int minPeriod() {
        int m = sz(P); if (m == 0) return 0;
        int len = m-pi[m-1]; if (m%len == 0) return len;
        return m;
    }
};
```

7.4 Manacher

Usage: Find all palindromic substrings in linear time.

Time Complexity: $O(N)$

```
// 각 문자를 중심으로 하는 가장 팰린드롬의 반경을 반환
// Manacher("abaaba") = {0,1,0,3,0,1,6,1,0,3,0,1,0}
// # a # b # a # a # b # a #
// 0 1 0 3 0 1 6 1 0 3 0 1 0
struct Manacher {
    vector<int> p;
    template<class T> Manacher(const T &s) {
        int n = sz(s)*2+1, c = 0, r = 0;
        p.assign(n, 0); T t; t.resize(n);
        for (int i = 0; i < sz(s); i++) t[i*2+1] = s[i];
        for (int i = 0; i < n; i++) {
            if (r > i) p[i] = min(r-i, p[2*c-i]);
            while (i-p[i]-1 >= 0 && i+p[i]+1 < n && t[i-p[i]-1] == t[i+p[i]+1]) p[i]++;
            if (i+p[i] > r) c = i, r = i+p[i];
        }
    }
    bool chk(int l, int r) {
        if (l > r) return false;
        return p[l+r+1] >= r-l+1;
    }
    int cnt() {
```

```
    int res = 0;
    for (auto i : p) res += (i+1)/2;
    return res;
}
```

7.5 Suffix Array

Usage: Suffix array and LCP array.

Time Complexity: $sa : O(N \log N), lcp : O(N)$

```
// Suffix Array - 모든 접미사 사전순 정렬 + LCP
// sa[i] : i번째 사전순 작은 접미사의 시작 인덱스 (0-idx)
// lcp[i] : sa[i-1] ~ sa[i] 공통 접두사 길이 (lcp[0]=0)
struct SA {
    int n;
    vector<int> sa, lcp, x, y, c;
    SA(const string& s) : n(sz(s)), sa(n), lcp(n), x(2*n, -1),
        y(2*n, -1), c(max(n, 256)) {
        for (int i = 0; i < n; i++) c[x[i] = s[i]]++;
        for (int i = 1; i < sz(c); i++) c[i] += c[i-1];
        for (int i = n - 1; i >= 0; i--) sa[--c[x[i]]] = i;
        for (int d = 1, p = 0; ; d <= 1, c.resize(p+1), p = 0) {
            for (int i = n - d; i < n; i++) y[p++] = i;
            for (int i = 0; i < n; i++) if (sa[i] >= d) y[p++] =
                sa[i] - d;
            fill(all(c), 0);
            for (int i = 0; i < n; i++) c[x[y[i]]]++;
            for (int i = 1; i < sz(c); i++) c[i] += c[i-1];
            for (int i = n - 1; i >= 0; i--) sa[--c[x[y[i]]]] =
                y[i];
            swap(x, y); p = x[sa[0]] = 0;
            for (int i = 1; i < n; i++)
                x[sa[i]] = (y[sa[i-1]] == y[sa[i]] && y[sa[i-1]+d] ==
                    y[sa[i]+d]) ? p : ++p;
            if (p == n-1) break;
        }
        for (int i = 0, k = 0; i < n; i++, k = max(0, k-1)) {
            if (x[i] == 0) continue;
            for (int j = sa[x[i]-1]; s[i+k] == s[j+k]; k++);
            lcp[x[i]] = k;
        }
    }
};
```

7.6 Z-algorithm

Usage: Longest common prefix between S and its suffixes.

Time Complexity: $O(N)$

```
// Z[i] = LongestCommonPrefix(S[0:N], S[i:N])
// = S[0:N]과 S[i:N]이 앞에서부터 몇 글자 겹치는지
vector<int> Z(const string &s) {
    int n = s.size();
    vector<int> z(n);
    z[0] = n;
    for (int i=1, l=0, r=0; i<n; i++){
        if (i < r) z[i] = min(r-i-1, z[i-l]);
        while (i+z[i] < n && s[i+z[i]] == s[z[i]]) z[i]++;
        if (i+z[i] > r) r = i+z[i], l = i;
    }
    return z;
}
```

7.7 Eertree

Usage: Manages all distinct palindromic substrings using length and suffix links.

Time Complexity: $O(N \log \Sigma)$

```
// Eertree(palindromic tree): "서로 다른 팰린드롬 부분
문자열"을 노드로 압축.
// 각 노드 = 팰린드롬 하나. 간선 = 양끝에 같은 문자 붙여 만든
더 긴 팰린드롬.
// "distinct 팰린드롬" 개수·등장 횟수
// for (char c : s) et.add(c); // 문자열 순서대로 삽입
(온라인)
// et.count(); // 등장 횟수
// 서로 다른 팰린드롬 개수 = sz(tree) - 2 // 노드 0,1은
가상/빈 루트
// tree[i].l = 그 팰린드롬 길이
// tree[i].c = 등장 횟수 (count() 후에만 정확)
// tree[i].f = suffix link (가장 긴 진짜 접미사 팰린드롬)
struct EERTREE {
    struct Node {
        int l, f, c, p, nxt[26];
        Node(int l, int f, int p = 0) : l(l), f(f), c(0), p(p) {
            for (int i = 0; i < 26; i++) nxt[i] = 0;
        }
    };
    vector<Node> tree; string s; int last;
    void reset() { s = "#"; last = 1; }
    EERTREE() { reset();
        tree.push_back({ -1, 0 }); tree.push_back({ 0, 0 });
    }
    int find(int x) {
        int p = sz(s)-1;
        while (s[p - tree[x].l - 1] != s[p]) x = tree[x].f;
        return x;
    }
    void add(char c) {
        s += c; int v = c-'a', p = find(last);
        if (!tree[p].nxt[v]) {
            int lnk = (p == 0 ? 1 : tree[find(tree[p].f)].nxt[v]);
            tree.push_back({ tree[p].l+2, lnk, p });
            tree[p].nxt[v] = sz(tree)-1;
        }
        tree[last = tree[p].nxt[v]].c++;
    }
    void count() {
        for (int i = sz(tree)-1; i > 1; i--) tree[tree[i].f].c
            += tree[i].c;
    }
};
```

7.8 Suffix Automaton

Usage: Builds a DFA representing all substrings. Supports substring counting and pattern matching.

Time Complexity: $O(N \cdot \Sigma)$

```
// 문자열의 모든 부분 문자열을 상태(≤2N)로 압축해 담은 DFA
// 부분 문자열의 개수·등장 횟수·매칭. 0(len)
// suffix_automaton<int, initialized_array<int,26,0>> T;
// T.extend(s); // 문자열을 순서대로 삽입
// T.ans // 서로 다른 부분 문자열 개수
// T.next[v][c] // 전이 (없으면 0), v=0이 루트
// T.len[v], T.link[v] // 상태 길이, suffix link
// * 등장 횟수: 각 상태의 endpos 크기 = "그 상태의 가장 긴
문자열" 등장 횟수
// extend 후, 각 상태 cnt=1(클론은 0)로 두고 link 트리에서
자식->부모로 누적함
template<typename T, size_t S, T init_val>
struct initialized_array : public array<T, S> {
    initialized_array() { this->fill(init_val); }
};
template<class Char_Type, class Adjacency_Type>
struct suffix_automaton{
    // Begin States
    // len: length of the longest substring in the class
    // link: suffix link
    // firstpos: minimum value in the set endpos
    vector<int> len{0}, link{-1}, firstpos{-1}, is_clone{false};
    vector<Adjacency_Type> next{};
    ll ans{0LL}; // 서로 다른 부분 문자열 개수
    // End States
    void set_link(int v, int lnk){
        if(link[v] != -1) ans -= len[v] - len[link[v]];
        link[v] = lnk;
        if(link[v] != -1) ans += len[v] - len[link[v]];
    }
    int new_state(int l, int sl, int fp, bool c, const
Adjacency_Type &adj){
        int now = len.size(); len.push_back(l);
        link.push_back(-1);
        set_link(now, sl); firstpos.push_back(fp);
        is_clone.push_back(c); next.push_back(adj); return now;
    }
    int last = 0;
    void extend(const vector<Char_Type> &s){
        last = 0; for(auto c: s) extend(c); }
    void extend(Char_Type c){
        int cur = new_state(len[last] + 1, -1, len[last], false,
{}), p = last;
        while(~p && !next[p][c]) next[p][c] = cur, p = link[p];
        if(!p) set_link(cur, 0);
        else{
            int q = next[p][c];
            if(len[p] + 1 == len[q]) set_link(cur, q);
            else{
                int clone = new_state(len[p] + 1, link[q],
firstpos[q], true, next[q]);
                while(~p && next[p][c] == q) next[p][c] = clone, p =
link[p];
                set_link(cur, clone); set_link(q, clone);
            }
        }
        last = cur;
    }
    int size() const { return (int)len.size(); } // # of states
}; suffix_automaton<int, initialized_array<int,26,0>> T;
```

```
// for(auto c : s) if((x=T.next[x][c]) == 0) return false;
```

8 STL & pbds

8.1 Hash map (pb_ds)

Usage: Faster hash table using pb_ds.

Time Complexity: $O(1)$

```
// faster than unordered_map, cannot use .count()
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
gp_hash_table<int,int> hashmap;
gp_hash_table<int,null_type> hashset;
```

8.2 Ordered Set (pb_ds)

Usage: Set supporting order_of_key and find_by_order.

Time Complexity: $O(\log N)$

```
// k번째 원소확인 및 x보다 작은 원소개수확인  $O(\log N)$ 
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template <typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;
// ordered_set<int> os;
// os.find_by_order(k): k번째 원소의 iter 반환 (0-idx, 없으면
os.end())
// os.order_of_key(x) : x보다 작은 원소의 개수 반환
template <typename T>
using ord_multiset = tree<T, null_type, less_equal<T>,
rb_tree_tag, tree_order_statistics_node_update>;
auto m_find(ord_multiset<int> &os, int val) {
    auto it = os.find_by_order(os.order_of_key(val));
    if (it != os.end() && *it == val) return it;
    return os.end();
} // os.erase(m_find(os, val))
```

8.3 Permutation & Combination

Usage: next_permutation and mask-based combinations.

```
#include <algorithm>
/* 1. Permutation */
sort(all(v))
do {
    // process v
} while (next_permutation(all(v)));
/* 2. Combination (nC_r): Use a mask vector */
vector<int> mask(n, 0);
fill(mask.end()-r, mask.end(), 1); // pick r elements
do {
    for (int i = 0; i < n; i++) {
        if (mask[i]) { /* v[i] is selected */ }
    }
} while (next_permutation(all(mask)));
/* 3. Partial Permutation (nPk) */
sort(all(v));
do {
    for(int i = 0; i < k; i++) { /* use v[i] */ }
    reverse(v.begin()+k, v.end());
} while (next_permutation(all(v)));
```

8.4 Priority Queue (pb_ds)

Usage: Meldable heap supporting modify/erase via point_iterator.

Time Complexity: $O(\log N)$

```
// 큐 병합, 임의 값 수정 및 삭제 가능
#include <ext/pb_ds/priority_queue.hpp>
using namespace __gnu_pbds;
template <typename T>
using pbds_pq = __gnu_pbds::priority_queue<T, less<T>,
pairing_heap_tag>;
int main() {
    pbds_pq<int> pq1, pq2;
    auto it = pq1.push(10); pq2.push(100);
    pq1.join(pq2); // 0(1), pq1: {10, 100}, pq2: {}
    pq1.modify(it, 50); // 0(logN), pq1 : {50, 100}
    pq1.erase(it); // 0(logN), pq1: {100}
    pq1.top(); pq1.empty(); pq1.size(); pq1.pop();
}
```

8.5 Rope

Usage: Persistent sequence supporting fast insertion, deletion and slicing.

Time Complexity: $O(\log N)$

```
#include <ext/rope>
using namespace __gnu_cxx;
int main() {
    string str; crope r(str.c_str()); // vector<T> v(N);
    rope<T> r(v.data(), N);
    r.insert(pos, str); r.erase(pos, len); // 0(logN)
    r.replace(pos, len, str); // 0(logN)
    crope r2 = r; // 0(1)
    r2 = r.substr(pos, len); // 0(logN)
    r += r2; // Append 0(logN)
    r[idx]; // 0(logN), but for(auto i : r) is 0(N)
    cout << r; // 0(N)
}
```

9 Misc

9.1 Custom Hash

Usage: Custom hash for pair, vector.

```
struct custom_hash {
    template <class T>
    void combine(size_t& seed, const T& v) const {
        seed ^= hash<T>{}(v) + 0x9e3779b9 + (seed<<6) + (seed>>2);
    }
    template <class T1, class T2>
    size_t operator()(const pair<T1, T2>& p) const {
        size_t seed = 0; combine(seed, p.first);
        combine(seed, p.second); return seed;
    }
    template <class T>
    size_t operator()(const vector<T>& v) const {
        size_t seed = 0;
        for (const auto& i : v) combine(seed, i);
        return seed;
    }
};
```


9.2 Fast I/O

Usage: Fast integer I/O using fread/fwrite.

```
struct FastIO {
    static constexpr int SZ = 1 << 16;
    char rb[SZ], wb[SZ];
    char *p1 = rb, *p2 = rb, *pp = wb;
    inline char getc() {
        return p1 == p2 && (p2 = (p1 = rb) + fread(rb, 1, SZ,
            stdin), p1 == p2) ? -1 : *p1++;
    }
    inline void putc(char c) {
        if (pp == wb + SZ) fwrite(wb, 1, SZ, stdout), pp = wb;
        *pp++ = c;
    }
    void read(auto &x) {
        x = 0; char c = getc(); bool f = 0;
        while (c != -1 && (c < '0' || c > '9')) { if (c == '-')
            f = 1; c = getc(); }
        while (c >= '0' && c <= '9') x = x * 10 + (c ^ 48), c =
            getc();
        if (f) x = -x;
    }
    void write(auto x) {
        if (x < 0) putc('-'), x = -x;
        static char t[20]; int ti = 0;
        do t[ti++] = (x % 10) ^ 48; while (x /= 10);
        while (ti) putc(t[--ti]);
    }
    ~FastIO() { fwrite(wb, 1, pp - wb, stdout); }
} io;
```

9.3 Random

Usage: Better random for mt19937.

```
#include <random>
#include <chrono>
mt19937_64
rng(chrono::steady_clock::now().time_since_epoch().count());
uniform_int_distribution<int>(1, r)(rng); // [1, r]
uniform_real_distribution<double>(1, r)(rng); // [1, r)
shuffle(all(v), rng); // shuffle vector
vector<double> w = { 40, 10, 50 };
discrete_distribution<int>(all(w))(rng); // 0: 40%, 1: 10%,
2: 50%
```

9.4 Ternary Search

Usage: Finding extremum of unimodal functions.

Time Complexity: $O(\log N)$

```
ll ternary_search(ll lo, ll hi, auto f) {
    if (lo > hi) return 0;
    while (hi - lo >= 3) {
        ll p = lo + (hi-lo) / 3, q = hi - (hi-lo) / 3;
        if (f(p) < f(q)) hi = q; // for max: f(p) > f(q)
        else lo = p;
    }
    ll res = lo; // for max: f(i) > f(res)
    for (ll i = lo+1; i <= hi; i++) if (f(i) < f(res)) res = i;
    return res;
}
```

```
}
double ternary_search(double lo, double hi, auto f, int
it=100) {
    while (it-->0) {
        double p = (lo*2 + hi) / 3., q = (lo + hi*2) / 3.;
        if (f(p) < f(q)) hi = q; // for max: f(p) > f(q)
        else lo = p;
    }
    return (lo+hi) / 2.;
}
```

9.5 Some tricks

Usage: Collection of bitwise hacks and optimization techniques.

```
__builtin_popcount(x); // 켜진 비트(1)의 총 개수
__builtin_clz(x); // 왼쪽(MSB)부터 연속된 0의 개수
__builtin_ctz(x); // 오른쪽(LSB)부터 연속된 0의 개수
// popcount를 유지하면서 다음으로 큰 수 / 작은 수
bool next_comb(ll& bit, int N) {
    ll x = bit & -bit, y = bit + x;
    bit = (((bit & ~y) / x) >> 1) | y;
    return (bit < (1LL << N));
}
ll init_comb(int n, int k) {
    return ((1LL << k) - 1) << (n - k);
}
bool prev_comb(ll& bit) {
    ll y = ~bit & ~~bit, x = bit & -y;
    bit = x - ((x & -x) / (y << 1));
    return x != 0;
}
// v(>0)보다 크고 popcount가 같은 가장 작은 정수
ll next_perm(ll v) {
    ll t = v | (v - 1);
    return (t+1)|(((~t & ~~t)-1)>>(__builtin_ctz(v)+1));
}
// mask의 모든 부분집합을 내림차순으로 순회 (0 제외), 0(3^N)
for (int sub = mask; sub > 0; sub = (sub-1)&mask);
// mask를 포함하는 모든 상위집합을 오름차순으로 순회
for (int sup = mask; sup < (1<<n); sup = (sup+1)&mask);
// 런타임 변수 n에 맞는 크기의 bitset을 사용
template <int len = 1> void solve(int n) {
    if (len < n) { solve<min(len*2, 200005)>(n); return; }
    bitset<len> bs;
    // do stuff
}
// bitset 고속순회 (켜져있는 비트만 순회)
for (int i = bs._Find_first(); i < bs.size(); i =
bs._Find_next(i)) {
    // do stuff
}
// 1부터 n까지의 수에서 숫자 i가 등장하는 총 횟수
ll count_digit_frq(ll n, int i) {
    ll ret = 0;
    for (ll j = 1; j <= n; j *= 10) {
        ll q = n / (j*10), r = n % (j*10);
        ret += (i == 0 ? (q-1)*j : q*j);
        if (r >= i*j) ret += (r < (i+1)*j ? r-i*j+1 : j);
    }
    return ret;
}
```

```
}
// 특정 날짜(년, 월, 일)의 요일 / 0: Sat, 1: Sun, ...
int get_day_of_week(int y, int m, int d) {
    if (m <= 2) y--, m += 12; int c = y / 100; y %= 100;
    int w = ((c>>2)-(c<<1)+y+(y>>2)+(13*(m+1)/5)+d-1)%7;
    if (w < 0) w += 7; return w;
}
// LIS
// a[i] < a[j] -> lower_bound
// a[i] <= a[j] -> upper_bound
vector<int> LIS(vector<int> v) {
    int n = sz(v);
    vector<int> lis, pos(n);
    for (int i = 0; i < n; i++) {
        auto it = lower_bound(all(lis), v[i]);
        pos[i] = it - lis.begin();
        if (it == lis.end()) lis.push_back(v[i]);
        else *it = v[i];
    }
    vector<int> res;
    for (int i = n-1, j = sz(lis)-1; i >= 0; i--) {
        if (pos[i] == j) {
            res.push_back(v[i]); j--;
        }
    }
    reverse(all(res));
    return res;
}
// 3D LIS
struct Cand {
    map<int,int> m; // y -> 그 y에서의 최소 z
    bool chk(int y, int z) const {
        auto it = m.lower_bound(y);
        return it != m.begin() && prev(it)->second < z; //
        non-strict: <=
    }
    void add(int y, int z) {
        auto it = m.lower_bound(y);
        if (it != m.begin() && prev(it)->second <= z) return;
        while (it != m.end() && it->second >= z) it = m.erase(it);
        m.insert(it, {y, z});
    }
};
int LIS3D(vector<array<int,3>> v) {
    sort(all(v), [](auto &a, auto &b) {
        if (a[0] != b[0]) return a[0] < b[0];
        return a[1] > b[1]; // non-strict: <
    });
    vector<Cand> res;
    for (auto [x, y, z] : v) {
        int lo = 0, hi = sz(res);
        while (lo < hi) {
            int mid = (lo+hi) / 2;
            if (res[mid].chk(y, z)) lo = mid + 1;
            else hi = mid;
        }
        if (lo == sz(res)) res.emplace_back();
        res[lo].add(y, z);
    }
}
```



```

    return sz(res);
}
// 도달 가능 여부 0(N^3 / 64)
bitset<MAXN> reach[MAXN];
for (int k = 0; k < n; k++) {
    for (int i = 0; i < n; i++) {
        if (reach[i][k]) reach[i] |= reach[k];
    }
}

```

10 Checklist + Useful Info

10.1 Highly Composite Numbers, Large Prime

< 10^k	number	divs	2	3	5	7	11	13	17	19	23	29	31	37
1	6	4	1	1										
2	60	12	2	1	1									
3	840	32	3	1	1	1								
4	7560	64	3	3	1	1								
5	83160	128	3	3	1	1	1							
6	720720	240	4	2	1	1	1	1						
7	8648640	448	6	3	1	1	1	1	1					
8	73513440	768	5	3	1	1	1	1	1	1				
9	735134400	1344	6	3	2	1	1	1	1	1				
10	6983776800	2304	5	3	2	1	1	1	1	1	1			
11	97772875200	4032	6	3	2	2	1	1	1	1	1			
12	963761198400	6720	6	4	2	1	1	1	1	1	1	1		
13	9316358251200	10752	6	3	2	1	1	1	1	1	1	1	1	
14	97821761637600	17280	5	4	2	2	1	1	1	1	1	1	1	
15	866421317361600	26880	6	4	2	1	1	1	1	1	1	1	1	1
16	8086598962041600	41472	8	3	2	2	1	1	1	1	1	1	1	1
17	74801040398884800	64512	6	3	2	2	1	1	1	1	1	1	1	1
18	897612484786617600	103680	8	4	2	2	1	1	1	1	1	1	1	1

< 10^k	prime	# of prime	< 10^k	prime
1	7	4	10	9999999967
2	97	25	11	99999999977
3	997	168	12	999999999989
4	9973	1229	13	9999999999971
5	99991	9592	14	9999999999973
6	999983	78498	15	99999999999989
7	9999991	664579	16	999999999999937
8	99999989	5761455	17	999999999999997
9	999999937	50847534	18	9999999999999989

10.2 Useful Stuff

- Catalan Number
 - 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900
 - $C_n = \binom{2n}{n} / (n + 1)$;
 - $C_n = \sum_{i=0}^{n-1} C_i \times C_{n-1-i}$;
 - 길이가 2n인 올바른 괄호 수식의 수
 - n + 1개의 리프를 가진 풀 바이너리 트리의 수
 - n + 2각형을 n개의 삼각형으로 나누는 방법의 수
 - n*n 격자에서 0,0부터 n,n까지 이동할 때, y=x(대각선)를 넘지않고 우 상단으로만 이동하는 경로의 수
 - n개의 노드로 이루어진 이진 트리 구조 개수
 - 1부터 n까지의 수를 스택에 넣고 빼서 만들 수 있는 순열의 개수

- 원 위에 2n개의 점이 있고, 2개씩 짝지어 선분을 그었을때 어떤 선분도 교차하지 않는 방법의 수
- (0, 0)에서 (N, M)까지 우상단으로 이동할 때, 직선 $y = x + k (k > 0)$ 에 닿지않는 경로의 수: 전체 경로 수 - 반사된 경로 수 = $\binom{N+M}{N} - \binom{N+M}{N-k}$ (직선 $y = x + k$ 에 처음 닿은 이후 경로를 $y = x + k$ 에 대해 대칭 이동-> $(M - k, N + k)$ 로 매핑됨)
- Derangements
 - N명의 사람이 모자를 벗어두었다가 무작위로 하나씩 집었을 때, 아무도 자기 모자를 쓰지 않을 경우의 수
 - 포함배제 ($D_N = N! \sum_{i=0}^N \frac{(-1)^i}{i!}$)- 팩토리얼 역원 전처리로 O(N)
 - DP ($D_N = (N - 1) \times (D_{N-1} + D_{N-2})$ (단, $D_0 = 1, D_1 = 0$))- O(N)
 - dp 테이블을 만들어두면 쿼리 당 O(1)
 - 부분 교환 순열: N명 중 정확히 K명만 자기 모자를 쓰고, 나머지 N - K 명은 아무도 자기 모자를 쓰지 않을 경우의 수 - $\binom{N}{K} \times D_{N-K}$
- Stars and Bars
 - N개의 구별되지 않는 물건을 K개의 구별되는 상자에 나누어담는 경우의 수 (빈 상자 허용)- $\binom{N+K-1}{K-1}$
 - $x_1 + x_2 + \dots + x_K = N$ ($x_i \geq 0$)의 해의 개수를 구하는 것과 동일
 - 각 상자에 적어도 1개 이상 담아야 한다($x_i \geq 1$): 미리 1개씩 나눠줌. $N' = N - K$ 로 치환해 $\binom{N'+K-1}{K-1} = \binom{N-1}{K-1}$ 계산
 - 어떤 상자도 C개를 초과할 수 없다($x_i \leq C$): 포함배제와 결합. (조건을 위반하는 상자를 1개, 2개 선택하는 경우를 빼고 더함)
- 제2종 스티링 수
 - 서로 다른 N개의 물건을 구별되지 않는 K개의 상자에 빈 상자 없이 나누어 담는 경우의 수 ($S(N, K)$ 혹은 $\left\{ \begin{matrix} N \\ K \end{matrix} \right\}$)
 - 점화식: $S(N, K) = K \times S(N - 1, K) + S(N - 1, K - 1)$ (단, $S(N, 1) = S(N, N) = 1$)
 - 직접 계산: $S(N, K) = \frac{1}{K!} \sum_{i=0}^K (-1)^{K-i} \binom{K}{i} i^N$ (단일 행 계산 시 $O(K \log N)$)
 - Bell Number ($B_N = \sum_{k=1}^N S(N, k)$): 서로 다른 N개의 원소를 분할하는 모든 방법의 수.
 - Bell Number 초항: 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975
- 자연수의 분할
 - 서로 같은 N개의 물건을 구별되지 않는 K개의 상자에 빈 상자 없이 나누어 담는 경우의 수 ($P(N, K)$)
 - 자연수 N을 K개의 자연수의 합으로 나타내는 방법의 수와 동일.
 - 점화식: $P(N, K) = P(N - 1, K - 1) + P(N - K, K)$
 - $P(N) = \sum_{k=1}^N P(N, k)$: 자연수 N의 총 분할 수.
 - P(N) 초항: 1, 1, 2, 3, 5, 7, 11, 15, 22, 30, 42, 56, 77, 101, 135
- Cayley’s Formula & Prüfer Sequence
 - N개의 구별되는 정점으로 만들 수 있는 서로 다른 스패닝 트리의 개수 = N^{N-2}
 - 각 정점의 차수가 $d_1, d_2, \dots, d_N$ 으로 고정되었을 때 가능한 트리의 개수 = $\frac{(N-2)!}{(d_1-1)!(d_2-1)! \dots (d_N-1)!}$ (단, $\sum d_i = 2N - 2$)
- Binomial Identities
 - 시그마가 포함된 조합론 수식을 O(1)로 단순화할 때 사용.
 - Hockey-stick Identity (파스칼의 삼각형 대각선 합): $\sum_{i=K}^N \binom{i}{K} = \binom{N+1}{K+1}$
 - Vandermonde’s Identity (두 집합에서 뽑기): $\sum_{i=0}^K \binom{N}{i} \binom{M}{K-i} = \binom{N+M}{K}$
 - 제곱의 합: $\sum_{i=0}^N \binom{N}{i}^2 = \binom{2N}{N}$

- Fibonacci Identities
 - Cassini’s identity: $F_{n-1}F_{n+1} - F_n^2 = (-1)^n$
 - Addition formula: $F_{n+k} = F_kF_{n+1} + F_{k-1}F_n$
 - Zeckendorf’s theorem: 모든 양의 정수는 연속하지 않는 피보나치 수들의 합으로 유일하게 표현 가능하며, 이는 그리디하게 큰 피보나치 수부터 빼는 것으로 구할 수 있다.
- Möbius Inversion
 - $g(n) = \sum_{d|n} f(d) \iff f(n) = \sum_{d|n} \mu(d)g(n/d)$
 - $g(n) = \sum_{n|d} f(d) \iff f(n) = \sum_{n|d} \mu(d/n)g(d)$
- Burnside’s Lemma
 - 경우의 수를 세는데, 특정 transform operation(회전, 반사, ..) 해서 같은 경우들은 하나로 친다. 전체 경우의 수는? 각 operation마다 이 operation을 했을 때 변하지 않는 경우의 수를 센다 (단, “아무것도 하지 않는다”라는 operation도 있어야 함!) 전체 경우의 수를 더한 후, operation의 수로 나눈다. (답이 맞다면 항상 나누어 떨어져야 한다)
- 알고리즘 게임
 - Nim Game의 해법: 각 더미의 돌의 개수를 모두 XOR했을 때 0이 아니면 첫번째, 0이면 두번째 플레이어가 승리.
 - Grundy Number: 어떤 상황의 Grundy Number는, 가능한 다음 상황들의 Grundy Number를 모두 모은 다음, 그 집합에 포함되지 않는 가장 작은 수가 현재 state의 Grundy Number가 된다. 만약 다음 state가 독립된 여러개의 state들로 나뉘는 경우, 각각의 state의 Grundy Number의 XOR 합을 생각한다.
 - Subtraction Game: 한 번에 k 개까지의 돌만 가져갈 수 있는 경우, 각 더미의 돌의 개수를 k + 1로 나눈 나머지를 XOR 합하여 판단한다.
 - Index-k Nim: 한 번에 최대 k개의 더미를 골라 각각의 더미에서 아무렇게나 돌을 제거할 수 있을 때, 각 binary digit에 대하여 합을 k + 1로 나눈 나머지를 계산한다. 만약 이 나머지가 모든 digit에 대하여 0이라면 두번째, 하나라도 0이 아니면 첫번째 플레이어가 승리. - Misere Nim: 모든 돌 무더기가 1이면 N이 홀수일 때 후공 승, 그렇지 않은 경우 XOR 합 0이면 후공 승
- Pick’s Theorem
 - 격자점으로 구성된 simple polygon이 주어짐. I 는 polygon 내부의 격자점 수, B 는 polygon 선분 위 격자점 수, A는 polygon의 넓이라고 할 때, 다음과 같은 식이 성립한다. $A = I + B/2 - 1$
- 가장 가까운 두 점: 분할정복으로 가까운 6개의 점만 확인
- 홀의 결혼 정리: 이분그래프(L-R)에서, 모든 L을 매칭하는 필요충분 조건 = L에서 임의의 부분집합 S를 골랐을 때, 반드시 (S의 크기) ≤ (S와 연결되어있는 모든 R의 크기)이다.
- 소수: 10 007 , 10 009 , 10 111 , 31 567 , 70 001 , 1 000 003 , 1 000 033 , 4 000 037 , 99 999 989 , 999 999 937 , 1 000 000 007 , 1 000 000 009 , 9 999 999 967 , 99 999 999 977
- 소수 개수: (1e5 이하 : 9592), (1e7 이하 : 664 579), (1e9 이하 : 50 847 534)
- 10¹⁵ 이하의 정수 범위의 나눗셈 한번은 오차가 없다.
- N의 약수의 개수 = $O(N^{1/3})$, N의 약수의 합 = $O(N \log \log N)$
- $\phi(mn) = \phi(m)\phi(n), \phi(pr^n) = pr^n - pr^{n-1}, a^{\phi(n)} \equiv 1 \pmod n$ if coprime
- Euler characteristic : v - e + f (면, 외부 포함) = 1 + c (컴포넌트)
- Euler’s phi $\phi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right)$

- Lucas’ Theorem $\binom{m}{n} = \prod \binom{m_i}{n_i} \pmod p$ m_i, n_i 는 p^i 의 계수
- 스케줄링에서 데드라인이 빠른 걸 쓰는게 이득. 늦은 스케줄이 안들어갈 때 가장 시간 소모가 큰 스케줄 1개를 제거하면 이득.
- 2차원 누적합 / imos (i = 행, j = 열, 1-based)
누적합 생성: $S[i][j] = S[i-1][j] + S[i][j-1] - S[i-1][j-1] + A[i][j]$
구간합 $[i_1, j_1] \sim [i_2, j_2]$: $S[i_2][j_2] - S[i_1-1][j_2] - S[i_2][j_1-1] + S[i_1-1][j_1-1]$
imos (+v), 부호 패턴 +, -, -, +:
 $d[i_1][j_1] += v, d[i_2+1][j_1] -= v, d[i_1][j_2+1] -= v, d[i_2+1][j_2+1] += v$
(imos는 마지막에 누적합 2번 적용: 행 방향, 열 방향)
- 순열 swap 관련
 - 버블소트 최소 swap = inversion 개수
 - 순열 정렬 최소 swap = $N - (\text{사이클 개수})$

10.3 자주 쓰이는 문제 접근법

- 비슷한 문제를 풀어본 적이 있던가?
- 단순한 방법에서 시작할 수 있을까? (brute force)
- 내가 문제를 푸는 과정을 수식화할 수 있을까? (예제를 직접 해결해보면서)
- 문제를 단순화할 수 있을까? / 그림으로 그려볼 수 있을까?
- 수식으로 표현할 수 있을까? / 문제를 분해할 수 있을까?
- 뒤에서부터 생각해서 문제를 풀 수 있을까? / 순서를 강제할 수 있을까?
- 특정 형태의 답만을 고려할 수 있을까? (정규화)
- 구간을 통째로 가져간다: 플로우 + 적당한 자료구조 $(i, i+1, k, 0), (s, e, 1, w), (N, T, k, 0)$
- 말도 안 되는 것 / 당연히하다고 생각한 것 다시 생각해 보기
- 특수 조건을 꼭 활용 / 여사건으로 생각하기
- 계임이론 - 거울 전략 혹은 mex DP 연계
- 겁먹지 말고 경우 나누어 생각 / 해법에서 역순으로 가능한가?
- 딱 맞는 시간복잡도에 집착하지 말자 / 문제에 의미있는 작은 상수 이용
- 스물투라지, 트라이, 해싱, 루트질 같은 트릭 생각
- 너무 추상화하기보단 풀려야 하는 방식으로 생각하기
- 잘못된 방법으로 파고들지 말고 버리자 / 제발 터널 비전에 빠지지 말자
- 헬프 콜은 적극적으로 / 혼자 멘탈 나가지 않기

10.4 DP 최적화 접근

- C[i, j] = A[i] * B[j]이고 A, B가 단조증가, 단조감소이면 Monge
- l.r의 값들의 sum이나 min은 Monge
- 식 정리해서 일차(HT) 혹은 비슷한(MQ) 함수를 발견, 구현 힘들면 Li-Chao
- $a \leq b \leq c \leq d$ 에서 $A[a, c] + A[b, d] \leq A[a, d] + A[b, c]$
- Monge 성질을 보이기 어려우면 N^2 나이트 짜서 opt의 단조성을 확인하고 찍맞
- 식이 간단하거나 변수가 독립적이면 DP 테이블을 세그 위에 올려서 해결
- 침착하게 점화식부터 세우고 Monge 인지 판별

- Monge에 집착하지 말고 단조성이나 볼록성만 보여도 됨

10.5 Graph Matching(Graph with $|V| \leq 500$)

- **Game on a Graph** : s 에 토권이 있음. 플레이어는 각자의 턴마다 토권을 인접한 정점으로 옮기고 못 옮기면 짐.
 s 를 포함하지 않는 최대 매칭이 존재함 $\leftrightarrow$ 후공이 이김
- **Chinese Postman Problem** : 모든 간선을 방문하는 최소 가중치 Walk를 구하는 문제.
Floyd를 돌린 다음, 홀수 정점들을 모아서 최소 가중치 매칭 (홀수 정점은 짝수 개 존재)
- **Unweighted Edge Cover** : 모든 정점을 덮는 가장 작은(minimum cardinality/weight) 간선 집합을 구하는 문제
 $|V| - |M|$, 길이 3짜리 경로 없음, star graph 여러 개로 구성
- **Weighted Edge Cover** : $sum_{v \in V}(w(v)) - sum_{(u,v) \in M}(w(u) + w(v) - d(u,v)), w(x)$ 는 x 와 인접한 간선의 최소 가중치
- **NEERC’18 B** : 각 기계마다 2명의 노동자가 다뤄야 하는 문제.
기계마다 두 개의 정점을 만들고 간선으로 연결하면 정답은 $|M| - |기계|$ 임. 정답에 1/2씩 기여한다는 점을 생각해보면 좋음.
- **Min Disjoint Cycle Cover** : 정점이 중복되지 않으면서 모든 정점을 덮는 길이 3 이상의 사이클 집합을 찾는 문제.
모든 정점은 2개의 서로 다른 간선, 일부 간선은 양쪽 끝점과 매칭되어야 하므로 플로우를 생각할 수 있지만 용량 2짜리 간선에 유량을 1만큼 흘릴 수 있으므로 플로우는 불가능.
각 정점과 간선을 2개씩 $((v, v'), (e_{i,u}, e_{i,v}))$ 로 복사하자. 모든 간선 $e = (u, v)$ 에 대해 e_u 와 e_v 를 잇는 가중치 w 짜리 간선을 만들고(like NEERC18), $(u, e_{i,u}), (u', e_{i,u}), (v, e_{i,v}), (v', e_{i,v})$ 를 연결하는 가중치 0짜리 간선을 만들자. Perfect 매칭이 존재함 $\Leftrightarrow$ Disjoint Cycle Cover 존재. 최대 가중치 매칭 찾은 뒤 모든 간선 가중치 합에서 매칭 빼면 됨.
- **Two Matching** : 각 정점이 최대 2개의 간선과 인접할 수 있는 최대 가중치 매칭 문제.
각 컴포넌트는 정점 하나/경로/사이클이 되어야 함. 모든 서로 다른 정점 쌍에 대해 가중치 0짜리 간선 만들고, 가중치 0짜리 (v, v') 간선 만들면 Disjoing Cycle Cover 문제가 됨. 정점 하나만 있는 컴포넌트는 self-loop, 경로 형태의 컴포넌트는 양쪽 끝점을 연결한다고 생각하면 편함.

10.6 Mincut 모델링

- N 개의 boolean 변수 $v_1, \dots, v_n$ 을 정해서 비용을 최소화하는 문제= true 인 점은 T , false 인 점은 F 와 연결되게 분할하는 민컷 문제
 1. v_i 가 T일 때 비용 발생: i 에서 F로 가는 비용 간선
 2. v_i 가 F일 때 비용 발생: i 에서 T로 가는 비용 간선
 3. v_i 가 T이고 v_j 가 F일 때 비용 발생: i 에서 j 로 가는 비용 간선
 4. $v_i \neq v_j$ 일 때 비용 발생: i 에서 j 로, j 에서 i 로 가는 비용 간선
 5. v_i 가 T면 v_j 도 T여야 함: i 에서 j 로 가는 무한 간선
 6. v_i 가 F면 v_j 도 F여야 함: j 에서 i 로 가는 무한 간선
- 5/6번 + v_i 와 v_j 가 달라야 한다는 조건이 있으면 MAX-2SAT
- Maximum Density Subgraph (NEERC’06H, BOJ 3611 팀의 난이도)
 1. density $\geq x$ 인 subgraph가 있는지 이분 탐색
 2. 정점 N 개, 간선 M 개, 차수 D_i 개
 3. 그래프의 간선마다 용량 1인 양방향 간선 추가

4. 소스에서 정점으로 용량 M , 정점에서 싱크로 용량 $M - D_i + 2x$
5. min cut에서 S와 붙어 있는 애들이 1개 이상이면 x 이상이고, 그제 subgraph의 정점들
6. while($r-l \geq 1.0/(n*n)$) 으로 해야 함. 너무 많이 돌리면 실수 오차

10.7 LP & Flow Duality

- **Difference Constraints** (차이 제약 $\rightarrow$ SPFA)
 - $x_v - x_u \leq w \iff u \rightarrow v$ (가중치 w).
 - $x_i \leq C \iff 0 \rightarrow i(C), x_i \geq C \iff i \rightarrow 0(-C).$
 - $x_i - x_j = C \iff$ 양방향 $(C, -C).$
 - 더미 노드 0을 시작점으로 SPFA 실행. 구한 최단 거리 dist[i] 값이 곧 제약을 만족하는 변수 x_i 의 해(최댓값).
 - 더미 노드 0 $\rightarrow i(w=0)$ 간선 필수. dist[u] == INF일 땐 relaxation 금지. cnt[u] >= V 감지 시 음수 사이클 존재 = 해 없음(모순).
- **Convex Potential $\rightarrow$ MCMF Dual** (포텐셜 차이 + 볼록 비용)
 - 제약 조건 $x_v - x_u \leq C \implies u \rightarrow v$ (용량 ∞ , 비용 C).
 - 목표 차이 T 및 오차 페널티 $|(x_v - x_u) - T|$: 정점 u, v 에 공급/수요 + T 부여 후 $u \leftrightarrow v$ 양방향 (용량 ∞ , 비용 1) 간선 생성.
 - 일반 볼록 비용 $f(x_v - x_u)$: (구간 길이 $\rightarrow$ 용량), (기울기 $\rightarrow$ 비용) 변환하여 $u \rightarrow v$ 다중 간선 생성.
 - MCMF 실행 $\rightarrow$ 최종 Min Cost가 구하는 최적 비용. (포텐셜 복원 시 잔여 그래프 h_u 활용)
 - 제약 간선 비용 $C < 0$ 이면 일반 Dijkstra MCMF 불가 $\rightarrow$ SPFA-Dijkstra MCMF 적용, 또는 Min-Cost Circulation 사용.
- **Planar Min-Cut $\rightarrow$ Dual Dijkstra** (평면 그래프 민컷)
 - 면을 정점으로 변환. 원본 간선의 좌/우 면을 잇는 Dual 간선 생성 (가중치 = 원본 용량).
 - S_{face} 에서 T_{face} 까지 다익스트라 실행. 최단 거리 $dist[T_{face}]$ = 원본 그래프의 최소 컷 값.
 - 외곽 면을 S_{face}, T_{face} 2개로 분리 정의. 용량 ∞ 원본 간선은 Dual 간선을 생성하지 않음.